Table of Contents

# FIRMWARE BUILD AND END-OF-LINE PROVISIONING

**Document:** FW-EEG-001 **Revision:** D **Date:** 2026-09-01 **Issued by:** TI One Voice research programme (one.witysk.org), Brussels, Belgium **Licence:** CC BY-SA 4.0 for this document; the firmware source is MIT **Governing documents:** DSN-EEG-003 Rev D, then RFQ-EEG-001 Rev E. Where this document and design.py disagree, design.py governs.

**Revision note (Rev C):** carries the layout findings – the carrier is now 150.0 x 130.0 mm and four layers – and **rules** the ring-buffer depth to a 6 MiB ring, 126 s at 1000 Hz, against the relaxed F-06 (ECO-EEG-025). That was a ruling, not a report of work done: when Rev C was issued `main.c` still asked for 12 MiB, which was FW-D13 and open. The source has since been changed and FW-D13 is closed – see the correction note below. It corrects the frame bandwidth to 50.7 kB/s, the end-of-line flashing route to the DevKitC-1's own UART port, the E-28 position, the eFuse phasing and the contact-light driver status. The findings of the second cross-document audit of 1 September 2026 that name this document – the layout-rules citation in section 3.1, the ring-buffer arithmetic in section 5.8, and the microSD file layout, which this document now owns in section 5.9 – are closed in this issue. The revision letter does not move: these are corrections within one release, not a new one.

**Correction note, 2 September 2026.** The firmware source was changed on 2 September and this document is re-synchronised against it here. An independent review found that Rev C described the *v1* source rather than the shipped one, and that matters more than ordinary staleness: a specification is the contract a builder follows, and as issued this document told them to re-introduce defects the source no longer had. What changed, and where this document changed with it:

- `firmware/main/drivers.h` is **new**. `main.c` had been calling seven functions that `drivers.c` defines with no declaration in scope. An implicit declaration is an error in C99 and later, and ESP-IDF compiles with `-Werror=implicit-function-declaration`, so the project could not have been built as shipped; worse, where a compiler accepts it the return type is assumed to be `int`, which would have truncated `sd_free_mb()`'s `uint32_t` and misread the pointer returns with no diagnostic at all. Section 1.2.
- `main.c` now includes `board_pins.h`, `esp_attr.h` and `drivers.h`. **FW-D12 and FW-D13 are closed in the source**, not merely ruled: the Rev A pin block is gone and `RING_BYTES` is 6 MiB at `main.c:101`.
- **FW-D14 is fixed.** `rx_poll()` strips the ten-byte frame header and checks the protocol version and the frame type before dispatch. It had been passing the whole frame, so `handle_command()` read header byte 0 – the protocol version – as the opcode; since `PROTO_VERSION` is 1 and `CMD_START_SESSION` is `0x01`, every command the browser tool sent started a recording session. Sections 1.1 and 6.1.
- The CMD_ACK payload is reshaped to section 6.2 exactly, in `ack_emit()`. Section 6.2 also records the one path that does **not** yet use it – provisioning, now FW-D20.

- `CMD_IDENTIFY` sets `CAP_CODEC`, which nothing had ever set, so a working codec always read as absent; and it separates `CAP_ATECC` (the part is fitted and answering) from `CAP_PROVISIONED` (its configuration zone is locked), which had been conflated, so an unprovisioned board coming off the line reported no secure element at all. Section 6.3.
- Section 6.3's opcode table assigned `0x0F` to FW_UPDATE_END, colliding with the `CMD_IDENTIFY` that `main.c` and TOOL-EEG-022 have used since 1 September, and had no row at all for `0x10` `CMD_LOOPBACK`. The table now carries both commands and FW_UPDATE_END has moved to `0x18`.
- `webtest/tests/interop/` is **new**: it compiles the real `main.c` against small ESP-IDF stubs on a development host and drives it with the real browser-tool protocol module. Section 8. This was the first time any of this firmware had been compiled or executed at all, and it was not a build against ESP-IDF and not a run on hardware. **Superseded the same day** – see the build note immediately below.

**Build note, 2 September 2026.** The firmware was **built** later the same day, against a real ESP-IDF installation, and then **run under QEMU**. Both are new facts and both are narrower than they sound; the whole of this document is re-synchronised against them here, because a specification that still says "never compiled" after the compile is a document a reader has to second-guess.

- The build is **ESP-IDF v5.2.5**, target `esp32s3`, from `sdkconfig.defaults` plus `sdkconfig.phase1`. Section 2.1 pinned **v5.2.2** and is corrected. The four release images and a manifest of their SHA-256 are in `firmware/release/`; section 9 carries the figures.
- Getting there took five corrections to the **project**, not to the protocol, and each is recorded in the file it was made in: `esp_driver_i2c` does not exist at IDF 5.2 and the component list would not configure; `CONFIG_BOOTLOADER_APP_ANTI_ROLLBACK=y` had been appended to `sdkconfig.phase1` below the deliberate `=n`, where a later duplicate silently wins, and ESP-IDF refuses anti-rollback on a table carrying a `factory` partition, so the build stopped; `CONFIG_TINYUSB_VENDOR_ENABLED` is not an option name that exists in `esp_tinyusb` 1.4.x, so the vendor interface this device is built around was silently absent – the real key is `CONFIG_TINYUSB_VENDOR_COUNT`; and three descriptor callbacks were defined here **and** in `esp_tinyusb`, which is FW-D02, and the link failed until the three were deleted and the component left to own them; and the five `sdkconfig` lines this document had listed as "must be added before the first build" were still not added. Sections 2.2 and 2.4.
- **FW-D02, FW-D03, FW-D11 and FW-D16 are closed, and FW-D20 is closed.** The `_Static_assert` on the MS OS 2.0 descriptor length has now been evaluated against resolved macros and held; the bicolour contact-light phase driver is written, so **E-27 is met in the source** and the FW-D16 row is retired; and `handle_provision()` now answers through `ack_emit()`, so the provisioning family is the section 6.2 acknowledgement. Sections 1.3, 3.2 and 6.2.
- Two opcodes were added: **`0x4A` CMD_READ_CALIBRATION** and **`0x4B` CMD_ATECC_WRITE_CONFIG**. Section 6.3's table did not carry them and now does.
- Section 7.3's table said it was `provision.py`'s step list. It was not, and the divergence was not cosmetic: the script writes and locks the ATECC configuration

zone **before** GenKey, because a factory-fresh part will not generate a key into an unconfigured slot. That section is rewritten from the script.

- The QEMU run is in `firmware/release/qemu_boot.log`. **It is an emulator, not a unit**, and section 8.2 states exactly what it does and does not prove.
- The build produced two memory figures and one of them is a finding. The image is **405,245 bytes** in a 3 MB slot, which is ample. **Static IRAM is 16,383 of 16,384 bytes used – one byte free**, which is a cliff and not a margin, and taking two unused ISRs out of IRAM did not move it by a byte. Section 9 records both and section 10 item 17 carries the open one.
- Every line count and line-number citation in sections 1.1, 1.2 and 1.3 was stale by up to 130 lines and is re-counted; the interop harness reports **57 checks**, not the 32 section 8.1 quoted.

The revision letter does not move: nothing here is a new release. Two things a reader tracks **do** change, and they are called out rather than buried. The ESP-IDF pin moves from **v5.2.2 to v5.2.5**, which is the version that produced the shipped images, and anyone reproducing the build must use it. And **E-27 moves from not met to met in the source**, because the phase driver it was waiting on is written – met in the source is not signed off, and TST-EEG-004 T11 still needs a unit and a colorimeter. No pin, no part and no quotable quantity changes.

### Why this document exists

Package v1 shipped one file, `firmware/main.c`, and nothing around it. There was no project, no configuration, no partition table, no pin map that matched the board, no provisioning script and no host tool – yet RFQ-EEG-001 section 1.1 asks a manufacturer to price “firmware loading and provisioning at the end of the line (we supply the firmware image and the provisioning script)”, and TST-EEG-004 makes running that script a test step. This document specifies everything that was missing, states exactly what the source does and does not do today, and defines the byte-level contract that the firmware, the manufacturer’s test station and the programme’s browser session runner must all agree on.

*Corrected 2026-09-02.* This paragraph read “**Nothing in this package has been built, and no safety engineer has reviewed this design.** Nothing here has been compiled or run on hardware.” Half of that is no longer true and the half that is withdrawn is withdrawn here rather than deleted: the firmware **was** compiled, against ESP-IDF v5.2.5, on 2 September 2026, and the images are in `firmware/release/`. What still stands, unchanged and in full:

**No safety engineer has reviewed this design. No board has been fabricated and no unit exists.** The firmware has run only under QEMU, whose `esp32s3` machine emulates none of this instrument’s peripherals (section 8.2). Every number marked *calculated* is still calculated, and every register value, daisy order and SPI timing in this document is still an assumption that no silicon has answered.

---

## 1. What exists and what is stubbed

The reference source is no longer one file. It is `firmware/main/main.c` (832 lines), `firmware/main/drivers.c` (580 lines) and `firmware/main/drivers.h` (77 lines), over the generated `firmware/main/board_pins.h` (74 lines); counted with `wc -l` on the shipped files at 2 September 2026. *Corrected 2026-09-02: this paragraph said 705,*

*476 and 69, which were the counts before the phase driver, the two new provisioning opcodes and the acknowledgement rework landed.* `main.c` was 362 lines in v1, and the growth is the provisioning handler of section 7, the acknowledgement of section 6.2, the contact-light phase driver of section 3.2, the two connectivity commands of TOOL-EEG-022 and the receive-path fix.

**It builds.** *Corrected 2026-09-02: this paragraph used to say the source had "never been built against a real ESP-IDF installation".* It has been, once, on 2 September 2026: ESP-IDF **v5.2.5**, target `esp32s3`, `sdkconfig.defaults` + `sdkconfig.phase1`, clean at ESP-IDF's default `-Wall -Werror=all`. The four images and their SHA-256 are in `firmware/release/` and section 9 carries the figures. That closes the *build* half of the FW-D01 exit criterion and it retires nothing else.

Read the rest of the claim narrowly, because three different things have now happened to this source and they prove three different amounts:

| What ran | What it proves | What it does not |
|---|---|---|
| `sh webtest/tests/interop/run.sh` – `main.c` compiled on a development host against small ESP-IDF stubs and driven by the real browser-tool protocol module: **57 checks, all passing** | the firmware and the host tool agree about framing, the CRC, the header, the opcodes, the acknowledgement layout, the S-01 interlock and the `0x4A`/`0x4B` split | nothing about ESP-IDF, TinyUSB, the descriptors or any peripheral: the stubs expand the descriptor macros to placeholders |
| `idf.py build` against **ESP-IDF v5.2.5** | every ESP-IDF header, Kconfig option and TinyUSB macro this file names resolves; the `_Static_assert` on the 10-byte frame header and the one on the MS OS 2.0 descriptor length both held; the image links and fits | that any of it is *right*. A register value that assembles is not a register value the ADS1299 accepts |
| `qemu-system-xtensa -M esp32s3` – one full boot, `firmware/release/qemu_boot.log` | the image boots, the bootloader reads **this** partition table, the app loads from the `factory` slot, `app_main()` runs, and the microSD, codec and ring-buffer paths behave as written | every peripheral. QEMU's `esp32s3` has no octal PSRAM, no microSD, no ES8388 and no ADS1299. Section 8.2 |

It has still **never run on hardware**, so the daisy order, the SPI timing, every register value and the whole analogue front end remain assumptions. The two C++ range-based `for` loops that made the v1 file a syntax error in C are gone (FW-D01).

The following inventory is what a reader will find in the file today.

### 1.1 Implemented in full

Line numbers are `main.c` at 2 September 2026, **re-counted after the build**. *Corrected 2026-09-02: every line number in this table was stale, by up to 130 lines at the foot of the file.*

| Function | Lines | What it does | Confidence |
|---|---|---|---|
| `crc32()` | 115-119 | IEEE 802.3 reflected CRC-32, polynomial `0xEDB88320`, init `0xFFFFFFFF`, final inversion. Matches Python `binascii.crc32` | High – byte-checkable offline, and checked against the host implementation by the interop harness |
| `cobs_encode()` | 121-128 | Standard COBS with the trailing `0x00` delimiter written by the encoder | High – decoded by the browser tool in the interop harness |
| `frame_hdr_put()` | 147-159 | Serialises the ten-byte header field by field, with a `_Static_assert` on the length (FW-D19) | High – the harness parses the result with the host decoder, and the assert held in the ESP-IDF build |
| `frame_emit()` | 167-184 | Builds header, appends payload, appends CRC, COBS-encodes, writes to CDC, vendor, ring buffer and SD | Structurally complete, **not thread-safe** (see FW-D04) |
| `ads_cmd()`, `ads_wreg_all()`, `ads_set_rate()` | 187-198 | Single-byte opcodes and daisy-chain register writes | Register values plausible, never verified against silicon |
| `ads_init()` | 199-224 | SPI bus, reset sequence, CONFIG1/2/3, per-channel gain, lead-off, MISC1, RDATAC | Compiles and links; never run against a converter, in QEMU or anywhere else |
| `drdy_isr()` | 237-247 | Increments `g_sample_index`, latches the envelope-comparator level and records its rising edge (FW-D08), notifies the sample task. **The only place the counter moves** (E-19) | Correct in principle |
| `aux_bits()` | 248-257 | Builds six of the sixteen auxiliary bits | Incomplete, see section 5.3 |

| Function | Lines | What it does | Confidence |
|---|---|---|---|
| `sample_task()` | 258-282 | 54-byte daisy read, de-interleaves the two devices, packs 50-byte sample records, emits one frame every 20 ms | Never run; daisy order is an assumption |
| `lights_write()` | 285-288 | Shifts eight bits into the 74HC595 and latches | Never run |
| `lights_phase()`, `lights_task()` | 325-360 | **The bicolour phase driver (E-27).** Phase A lights the wanted sites green with `LED_V` high, phase B lights them red with `LED_V` low, a site in both is amber; the masks come from the converter's positive-side lead-off comparator read at **two thresholds** – trips neither is green, the sensitive one only is amber, both is red (FW-D17). Dark while recording, and `LED_V` is returned to an **input** rather than driven low, which is dark by construction | **Written, on 2 September 2026 – FW-D16 is closed and E-27 is met in the source.** Never run. The alternation quantises to the FreeRTOS tick: nominal `LIGHT_PHASE_HZ` 240, actual about **250 Hz** – section 3.2 |
| `ack_emit()` | 430-439 | The section 6.2 acknowledgement: opcode echoed, reserved zero, status, result length, result | High – decoded by the browser tool's own `parseAck()` in the harness |
| `handle_provision()` | 441-561 | The **thirteen** `0x40-0x4B` and `0x4F` provisioning opcodes of section 7, including `0x4A` CMD_READ_CALIBRATION and `0x4B` CMD_ATECC_WRITE_CONFIG | Never run against an ATECC608B. Its acknowledgement **is** the section 6.2 one as of 2 September – FW-D20 closed |
| `handle_command()` | 563-658 | Dispatches eleven general opcodes – `0x01-0x05`, `0x08-0x0B`, `0x0F`, `0x10` – and routes `0x40-0x4F` to `handle_provision()` | Five declared opcodes still fall to the unknown-opcode arm: FW-D10 |

| Function | Lines | What it does | Confidence |
|---|---|---|---|
| `timing_selftest()` | 664-670 | 40 tone bursts, insertion sort, median and p95 | The two functions it calls now exist in `drivers.c`; neither has driven a codec |
| USB descriptor tables, `tud_descriptor_bos_cb()` | 673-722 | Device, configuration, string and BOS descriptors. *Corrected 2026-09-02:* there are **not** four `tud_descriptor_*_cb` callbacks here any more. `esp_tinyusb` owns the device, configuration and string callbacks and is handed those three as **data** through `tinyusb_config_t`; defining them here as well is FW-D02 and it failed the link with three “multiple definition” errors. Only `tud_descriptor_bos_cb()` stays, because `esp_tinyusb` 1.4.x defines none and `tinyusb_config_t` has no field for one | Macro names now **resolved** against the pinned `esp_tinyusb` by the real build, and `MS_OS_20_DESC_LEN` asserted. Never enumerated by a host |
| `tud_vendor_control_xfer_cb()` | 723-734 | Serves the WebUSB URL and the MS OS 2.0 descriptor | Never run |
| `rx_poll()` | 736-763 | One pass of the receive loop: COBS-decode, check the CRC, **skip the ten-byte header and check the version and the frame type**, then dispatch (FW-D14). Factored out of `rx_task()` so a host harness can drive the real path rather than a copy of it | Exercised end to end by the interop harness |
| `rx_task()` | 765-770 | Calls `rx_poll()`, and yields for 2 ms when it read nothing | Never run |

| Function | Lines | What it does | Confidence |
|---|---|---|---|
| `status_task()` | 773-778 | One STATUS frame per second | Payload is **still** 12 bytes carrying four of the eight items F-09 names, with link type at offset 8 where section 5.4 puts uptime (FW-D09, open) |
| `app_main()` | 780-825 | GPIO setup, `drv_init_all()`, serial read, ring buffer with an explicit abort if PSRAM will not give it, TinyUSB install, ADS init, four tasks | Runs to completion in QEMU as far as the ring-buffer allocation, which QEMU cannot satisfy and which aborts with the named diagnostic FW-D13 added. Never run on a target |

### 1.2 The drivers are written now, and none of them has run

At v2.1 these six symbols were declared `extern` **inside function bodies** and defined nowhere in the package. They are now declared in `firmware/main/drivers.h` and defined in `firmware/main/drivers.c`, which is in SRCS (section 2.2). Writing a driver and proving one are different claims and only the first has happened: not one of these has addressed a real peripheral, so every I2C address, register write and timeout below is still an assumption, and the test steps in the last column still gate on hardware.

*Line numbers corrected 2026-09-02.*

| Symbol | Defined at | Driver | Called from | Blocks |
|---|---|---|---|---|
| `void sd_append(const uint8_t *, size_t)` | `drivers.c:163` | SDMMC, one-bit | `frame_emit()`, every frame | T14 |
| `uint32_t sd_free_mb(void)` | `drivers.c:175` | SDMMC | `status_task()`, `CMD_IDENTIFY` | T14, F-09 |
| `void atecc_serial_into(char *, size_t)` | `drivers.c:288` | ATECC608B I2C | `handle_provision()` at `0x43` | T5b, T6, F-04 |
| `uint8_t battery_percent(void)` | `drivers.c:107` | MAX17048 I2C | `status_task()` | F-09, E-22 |
| `void codec_play_tone_at(uint32_t)` | `drivers.c:234` | ES8388 I2S | `timing_selftest()` | T12, T13, T17 |

| Symbol | Defined at | Driver | Called from | Blocks |
|---|---|---|---|---|
| `int envelope_onset_after(uint32_t, int)` | `drivers.c:250` | signal | `timing_selftest()` | T12, T13 |

`drivers.c` also defines `unit_serial_into()`, the `drv_atecc_*` and `drv_nvs_*` groups section 7 needs – including `drv_atecc_write_config()` at `drivers.c:431` and `drv_nvs_get_blob()` at `drivers.c:552`, both added on 2 September for the two new provisioning opcodes – and `drv_codec_ready()` and `drv_atecc_present()`, which are what `CMD_IDENTIFY` reports as `CAP_CODEC` and `CAP_ATECC`.

**The QEMU run exercised three of these paths and answered a question about all of them.** `drv_sd_init()` timed out, `drv_codec_init()` timed out on ES8388 register 0x00, and both logged the failure and continued rather than aborting, which is what they were written to do. That is a bring-up behaviour test and nothing more: QEMU emulates neither part, so a timeout is the only answer it could ever have given, and no I2C address, register write or timeout below has yet been answered by a part.

`drivers.h` exists because it had to. `main.c` called seven of these functions with no declaration in scope, which is an error in C99 and later and which ESP-IDF rejects at `-Werror=implicit-function-declaration`; and where a compiler does accept an implicit declaration it assumes a return type of `int`, so `sd_free_mb()`'s `uint32_t` and the pointer returns would have been truncated or misread at runtime with no diagnostic. Include it from both files, so the compiler checks the definitions against the prototypes the callers use.

A seventh item is worse than a stub: the **block-signing task is not declared at all**. The closing comment says SIGNATURE frames are "emitted as FT_SIGNATURE by the sd/signing task (not shown)". Nothing emits them, so T16 – the only place the study's integrity mechanism is ever tested – has no input. That is unchanged at 2 September: `drivers.c` contains no SHA-256, no chained digest, no signing call and no such task, and F-08 is therefore unimplemented rather than merely untested.

### 1.3 The bring-up defect register

These are the defects found by reading the file, and – from 2 September 2026 – by building it. Each is a work item with an acceptance test. Anyone quoting firmware bring-up should quote against this table, not against "complete the stubs". Twenty entries; **fourteen are fixed or closed in the source and six are open**. *Corrected 2026-09-02: this line read "nine are fixed and eleven are open". FW-D02, FW-D03, FW-D11, FW-D16 and FW-D20 have closed since, four of them because the first real build forced them into the open.*

A row marked FIXED means the source no longer has the defect. It does not mean the acceptance test has been run: the acceptance column is the gate, and for most of these rows that gate is a unit on a bench, which does not exist. **The build gate, however, is now real** – `idf.py build` against ESP-IDF v5.2.5 completes clean at ESP-IDF's default `-Wall -Werror=all` – so where a row's acceptance was "build once", that row is genuinely retired and the column says so.

The six open rows are **FW-D04, FW-D05, FW-D06, FW-D07, FW-D09 and FW-D10**. Five of the six are frame-integrity or protocol-completeness items and none of them is

visible to a compiler; the sixth, FW-D09, is a wire-format shortfall a host decodes wrongly in silence.

| ID | Defect | Violates | Fix | Acceptance |
|---|---|---|---|---|
| FW-D01 | C++ range-based `for` in `app_main()`, twice | compiles as C | **FIXED**: explicit arrays and an index loop, `main.c:783-793` | **MET 2026-09-02.** `idf.py build` against ESP-IDF v5.2.5 completes clean at ESP-IDF's default `-Wall -Werror=all`. A full `-Wextra -Werror` build has not been demonstrated and is not what ESP-IDF sets |

| ID | Defect | Violates | Fix | Acceptance |
|---|---|---|---|---|
| FW-D02 | Descriptor callbacks defined **and** the same descriptors passed to `tinyusb_driver_install()` | link error | **FIXED 2026-09-02, and the predicted fix was the wrong one.** This row used to say “set `CONFIG_TINYUSB_DESC_CUSTOM=y` and keep the callbacks”. Doing both is exactly the defect: with `CONFIG_TINYUSB_DESC_CUSTOM=y` the descriptors are supplied as **data** through `tinyusb_config_t` and `esp_tinyusb` owns the callbacks. The link failed with three “multiple definition” errors against `descriptors_control.c` until `tud_descriptor_device_cb()`, `tud_descriptor_configuration_cb()` and `tud_descriptor_string_cb()` were deleted from `main.c`. `tud_descriptor_bos_cb()` stays, because the component defines none | **Build gate MET** (the image links). T5 for enumeration |

| ID | Defect | Violates | Fix | Acceptance |
|---|---|---|---|---|
| FW-D03 | The TinyUSB vendor class is not enabled, so `tud_vendor_*` has no backing interface | F-01 | **FIXED 2026-09-02, and the option name in this row was wrong.** `CONFIG_TINYUSB_VENDOR_ENABLED` **does not exist** in `esp_tinyusb` 1.4.x, and ESP-IDF does not fail on an unrecognised Kconfig key – it warns and carries on. So the setting looked present, `CFG_TUD_VENDOR` stayed 0, `tusb.h` declared none of the `tud_vendor_*` functions, and the WebUSB half of F-01 would simply not have existed. It surfaced as “implicit declaration of function `tud_vendor_mounted`” the first time the file met a real compiler. The real key is **`CONFIG_TINYUSB_VENDOR_COUNT=1`**, now in both `sdkconfig.defaults` and `sdkconfig.phase1` | **Build gate MET.** T5 for enumeration |
| FW-D04 | One static `txbuf`/`cobsbuf` pair written by four callers at priorities 23, 10, 5 and 4 with no mutex | frame integrity | a recursive mutex around `frame_emit()`, or per-task encode buffers | T14, 30 min, zero bad CRC |
| FW-D05 | The overflow path drops the oldest frames and emits no GAP | F-07 “silent loss is not permitted” | emit FT_GAP per section 5.5 | T15 |

| ID | Defect | Violates | Fix | Acceptance |
|---|---|---|---|---|
| FW-D06 | `CMD_RETRANSMIT` ignores its range and drains the ring destructively | F-06 | index the ring by sequence, copy without consuming | T15 complete recovery |
| FW-D07 | `g_seq++` from three tasks, non-atomic | frame ordering | increment under the FW-D04 mutex | T14 |
| FW-D08 | The comparator aux bit is sampled in the task, not latched in the DRDY ISR | E-12 sub-sample onset | **FIXED**: `drdy_isr()` latches the level and records the rising edge in `g_onset_sample`, and `aux_bits()` reports the latched value (`main.c:235-255`) | T12b |
| FW-D09 | STATUS carries battery, lead-off, SD free and link type only, in a 12-byte payload, and link type sits at offset 8 where section 5.4 puts uptime, so a host parsing per section 5.4 decodes it as part of uptime | F-09 (8 items) | the 24-byte, 10-field payload of section 5.4, which includes the ring depth T15 grades against (section 5.8) | T14, T21, T15 |

| ID | Defect | Violates | Fix | Acceptance |
|---|---|---|---|---|
| FW-D10 | **Open, and unchanged by the build.** `CMD_SET_GAIN`, `CMD_IMPEDANCE`, `CMD_PLAY_AT`, `CMD_FW_UPDATE_BEGIN` and `CMD_PROVISION` are still declared in the enum (`main.c:363-369`) with no case arm, so all five reach the `default:` arm. The status they now return is `0x01`, unknown opcode – **not** `0xFF` any more, and not the `0x0B` not-implemented that section 6.2 defines and that a host needs in order to tell a missing feature from a typo. Of the five, only `CMD_PROVISION` is answered elsewhere: provisioning has moved to the `0x40-0x4F` family, which `handle_provision()` implements in full, so `0x0E` should be deleted from the enum rather than written | F-10 | section 6; and return `0x0B`, not `0x01`, where the honest answer is “not implemented in this build” | T10, T11, T12, T17 |

| ID | Defect | Violates | Fix | Acceptance |
|---|---|---|---|---|
| FW-D11 | `_Static_assert(sizeof(ms_os_20_desc) == 0xB2)` never checked against resolved macros | F-02 | **CLOSED 2026-09-02**: the assert (`main.c:705`) was evaluated in the ESP-IDF v5.2.5 build against the pinned `esp_tinyusb` and held, as did the ten-byte header assert at `main.c:160`. Every TinyUSB macro this file names now resolves. What that does **not** settle is whether Windows accepts the descriptor set | T5 on Windows 11 |
| FW-D12 | `main.c` carried the **Rev A** pin defines: `PIN_SR_DATA 35`, `PIN_SR_CLK 36`, `PIN_SR_LATCH 37`, which are the octal PSRAM bus, so a unit flashed with it would have torn down the memory the ring buffer lives in | ECO-EEG-009 | **FIXED**: the block is deleted and `main.c` includes the generated `board_pins.h`, which comes from `design.py`, so the firmware and the board cannot disagree | T11 |

| ID | Defect | Violates | Fix | Acceptance |
|---|---|---|---|---|
| FW-D13 | `RING_BYTES` was 12 MiB; the -N16R8 carries 8 MiB of PSRAM in total, so the allocation returned NULL and the first DATA frame asserted inside `xRingbufferSend` | F-06 | **FIXED**: `RING_BYTES` is `6*1024*1024` at `main.c:101`, and `app_main()` now aborts with a named diagnostic if PSRAM will not give it. F-06 relaxed by ECO-EEG-025 – section 5.8. **The diagnostic has been seen to work**: it is the last line of `firmware/release/qemu_boot.log`, because QEMU has no PSRAM at all to allocate from | T15 |
| FW-D14 | `rx_task()` treated the first decoded byte as the opcode; the host tool sends a full frame header first. The consequence was worse than non-interoperation: byte 0 is the protocol version, `PROTO_VERSION` is 1 and `CMD_START_SESSION` is `0x01`, so **every** command from the browser tool started a recording session, and the one command that appeared to work did so by that coincidence | interoperability | **FIXED on 2 September 2026**: `rx_poll()` dispatches from `out + FRAME_HDR_BYTES` and refuses any frame whose version is not `PROTO_VERSION` or whose type is not `FT_CMD` (`main.c:609-636`) | 32 of 32 in `webtest/tests/interop`; a provisioning dry run against hardware is still outstanding |
| ~~FW-D15~~ | *Closed at Rev C.* The header said “written against RFQ-EEG-001 Rev B” while the package shipped Rev E | traceability | done: this document is headed **RFQ-EEG-001 Rev E** and section 5 is written against it | closed |

| ID | Defect | Violates | Fix | Acceptance |
|---|---|---|---|---|
| FW-D16 | The bicolour contact-light phase driver did not exist. `lights_write()` and `lights_task()` drove the 74HC595 on and off only; nothing alternated LED_V against the shift-register outputs, so no site could show red or amber | E-27, RFQ E-27's amber state | **FIXED 2026-09-02**: `lights_phase()` and `lights_task()` (`main.c:325-360`) drive LED_V (GPIO48) and Q0-Q7 in antiphase, and the three colours come from the converter's positive-side lead-off comparator read at **two thresholds** – trips neither is green, the sensitive threshold only is amber, both is red (as corrected by FW-D17, below; as first written this used `LOFF_STATP & LOFF_STATN`, and red was unreachable). **E-27 is met in the source.** The alternation quantises to the 1 kHz FreeRTOS tick and runs at about **250 Hz**, not the nominal 240; the requirement is "above 100 Hz" and 250 Hz meets it. Section 3.2 | **T11.** The driver is written, so T11 is no longer blocked by its absence; it still needs a unit and a colorimeter, and it has never been run |
| FW-D17 | **`CMD_START_SESSION` armed a session without ever reading `PIN_VBUS_DET`.** Only the CHG_CE half of S-01 existed, so a unit tethered to a mains-powered charger would record | S-01 | **FIXED at Rev C**: the handler refuses first and returns ack status `0x05`, per section 6.1 | T21, and the interlock half of T3 |

| ID | Defect | Violates | Fix | Acceptance |
|---|---|---|---|---|
| FW-D18 | `CMD_SET_RATE` issued WREG CONFIG1 while the converter was in RDATAC; the ADS1299 ignores register writes in that mode, so the rate silently did not change | E-02 | **FIXED**: START low, SDATAC, write CONFIG1, RDATAC, START high (`main.c:512-523`) | T2 at all three rates |
| FW-D19 | **The frame header was `memcpyd` onto the wire with `sizeof(frame_hdr_t)`, which is 12, not 10.** The struct is tail-padded to its 4-byte alignment, so two padding bytes sat between the header and the payload and every host parser written to section 5.1 – including this package's own `verify_stream.py` – misparsed every frame by two bytes | section 5.1 | **FIXED at Rev C**: the header is serialised field by field by `frame_hdr_put()`, with a `_Static_assert` on the 10-byte length. A wire format is a contract with other people's software and must never be a struct copy | T5b, and TOOL-EEG-022 step D4 |

| ID | Defect | Violates | Fix | Acceptance |
|---|---|---|---|---|
| FW-D20 | The provisioning acknowledgements were not the section 6.2 acknowledgement. `handle_provision()` built `{opcode, status, result...}` and called `frame_emit()` directly, while `ack_emit()` and every other reply send `{opcode, reserved 0, status, result length, result...}`. `provision.py` reads the status at absolute frame offset 12, which is payload offset 2, so it read the first *result* byte as the status: on `0x48` READ_PROVISION_STATE that byte is the config-zone lock flag, so an **unlocked** unit reported 0 and looked like a success – the one command whose whole job is to say whether provisioning happened answered “fine” for a blank part | section 6.2 | **FIXED 2026-09-02**: the whole `0x40-0x4F` family returns through `ack_emit()` (`main.c:561`), including the not-in-provisioning-mode refusal. Status codes are unchanged; only the envelope moved | T6. **The interop harness now covers the shape**: three of its 57 checks read a provisioning status at the section 6.2 offset, and one of them is a `0x02` bad-length refusal that would have decoded as something else under the old shape. A run against a real ATECC608B is still outstanding |

Exit criterion for the whole backlog: a clean build at `-Werror`, enumeration on Windows 11, macOS and Ubuntu (T5), and T13 reporting median <= 1 and p95 <= 2 samples. *Corrected 2026-09-02: the first of those three is now met* – the ESP-IDF v5.2.5 build is clean at `-Wall -Werror=all`. **The other two are not, and they are the ones that need a unit.** The host harness of section 8 is not that build and retires nothing; it is a guard against the firmware and the host tool drifting apart again. The QEMU run of section 8.2 retires nothing either: a boot on an emulator with none of this instrument’s peripherals cannot grade a single row above.

## 2. Build environment

### 2.1 Toolchain

*Corrected 2026-09-02.* This table pinned **v5.2.2**. The firmware was built with **v5.2.5** – the released images in `firmware/release/` and their `manifest.json` say so – and a document that pins a version nobody used is a document a second builder will not reproduce. The pin moves to the version that produced the shipped images. `v5.2.2` remains inside the `>=5.2,<5.4` range `main/idf_component.yml` permits, so nothing about the component resolution changes.

| Item | Value | Why it is pinned |
|---|---|---|
| ESP-IDF | **v5.2.5**, tag `v5.2.5`; `main/idf_component.yml` permits `>=5.2,<5.4` | The TinyUSB descriptor macros moved between releases (FW-D11), and this is the version the shipped images were built with |
| Managed component | `espressif/esp_tinyusb` `~1.4.2`, which **resolved to 1.4.5** in this build | Same reason. `firmware/dependencies.lock` records what actually resolved and is part of the release |
| Target | `esp32s3` | E-18, not substitutable |
| Host | Ubuntu 22.04 or Windows 11, Python 3.11 | The release container is Linux |
| Container | `espressif/idf:v5.2.5`, pinned by image digest | Reproducible build, section 9 |

**The build is not yet reproducible in the sense section 9 asks for.** The shipped images were built on a developer machine, not in the pinned container, and the boot log records the application version as `e91f9d58-dirty` – a working tree with uncommitted changes. Reproducing the SHA-256 list needs a clean tree and the container, and that has not been done.

An unpinned build is a silent difference between units, and section 9 of RFQ-EEG-001 exists because device identity must not become a confound in the study.

### 2.2 Repository layout

```
firmware/
  CMakeLists.txt            project eeg_field_kit, VERSION 0.2.0
  sdkconfig.defaults        the fleet configuration (section 2.4)
  sdkconfig.phase1          the Phase 1 overlay: no secure boot, no
eFuses (section 2.4)
  sdkconfig.qemu            the emulator overlay -- NOT a configuration
for any unit (8.2)
  partitions.csv            the flash layout (section 2.5)
  dependencies.lock         what the component manager actually
resolved
  README.md                 status and the three commands
  main/
    CMakeLists.txt          idf_component_register(...)
```

```
    idf_component.yml       pinned dependencies
    board_pins.h            GENERATED from tools/design.py (section 3)
    drivers.h               the peripheral API main.c calls (section
1.2)
    drivers.c               the implementations of it (section 1.2)
    main.c                  the reference source
  tools/
    provision.py            end-of-line provisioning (section 7)
    verify_stream.py        host verification (section 8)
    provision_selftest.py   offline checks on provision.py, including
opcode uniqueness
    atecc608b_config.py     generates the ATECC608B configuration-zone
template
  release/                  the built images, their manifest, the size
report and the
                            QEMU boot log (sections 8.2 and 9).  ADDED
2026-09-02
```

`build/` is a build directory, not a release artefact, and is not part of the package.

The firmware also has a test that is not under `firmware/`: `webtest/tests/interop/` compiles `main.c` against ESP-IDF stubs and drives it with the browser tool's protocol module. It lives next to the tool it proves agreement with, and section 8 describes it.

`tools/provision.py` and `tools/verify_stream.py` are the source-tree paths and are the names this package uses. Other documents in the release call the same two programs by other names; that divergence is recorded as an open item in section 10 and is not resolved here.

`main/CMakeLists.txt` as shipped:

```
idf_component_register(
    SRCS "main.c" "drivers.c"
    INCLUDE_DIRS "."
    # `driver` and NOT `esp_driver_i2c`.  The I2C driver was split into
its own component
    # at ESP-IDF 5.3; at 5.2, which idf_component.yml also allows,
`esp_driver_i2c` does
    # not exist and the build stops at configure with "component could
not be found".
    # `driver` provides I2C on both, so it is the requirement that
spans the pinned range.
    REQUIRES driver esp_timer esp_psram sdmmc fatfs esp_partition
app_update
             nvs_flash
    PRIV_REQUIRES esp_tinyusb
)
```

*Corrected 2026-09-02.* This block used to list `esp_driver_i2c` in `REQUIRES`. **That component does not exist at ESP-IDF 5.2** – the I2C driver was split out into its own component at 5.3 – and `idf_component.yml` permits `>=5.2,<5.4`, so the build stopped at configure with "component could not be found". `driver` provides I2C on both, so it is the requirement that spans the pinned range, and the line is now what is in the file.

The drivers came in as one `drivers.c` rather than the six files this section used to predict, and `nvs_flash` came in with them. What is still to be added is `signing.c` – the

block-signing task of section 1.2, which nothing implements – and an I2S component with it if the tone generator moves out of `drivers.c`.

### 2.3 Commands, and how a unit is put into download mode

```
. $IDF_PATH/export.sh                # ESP-IDF v5.2.5
idf.py set-target esp32s3
idf.py build                         # -> build/eeg_field_kit.bin
idf.py size                          # app size must stay under 3 MB
(section 2.5)
```

These are the commands that produced `firmware/release/` on 2 September 2026. The Phase 1 build – which is what was actually run, and what the two prototypes take – adds the overlay:

```
idf.py -DSDKCONFIG_DEFAULTS="sdkconfig.defaults;sdkconfig.phase1" build
```

`idf.py size` reported a **405,245-byte** image against a 3 MB slot, and static IRAM at **16,383 of 16,384 bytes used**. The second of those is an open item, not a pass; section 9 gives both figures and section 10 item 17 carries the IRAM one.

Bench and end-of-line flashing both go through the **DevKitC-1's own UART USB-C port**:

```
idf.py -p /dev/ttyUSB0 -b 921600 flash monitor
```

**That port is the only flashing route, and it is the route the production line uses.** The DevKitC-1 carries the auto-reset circuit on the module itself – the bridge's DTR and RTS lines drive EN and IO0 – so `esptool.py` puts the ESP32-S3 into download mode with no fixture, no relays and no operator action. The module is reachable with the pod lid off through the 31 x 61 mm opening in the MP-01 module plate, so the carrier does not have to be removed to flash a unit.

**The carrier's J26 header is console and recovery only and cannot enter download mode.** J26 carries 3V3, DGND, UART0 TX, UART0 RX, EN and one spare way netted `NC_GPIO0`. GPIO0 is committed to LED_SR_LATCH at J7 position 14 (ECO-EEG-009) and does not reach J26, so no relay sequence on J26 can assert IO0. Any procedure that describes flashing through J26 with a GPIO0/EN relay sequence is wrong. Console output is UART0 on GPIO43/44, brought to J26, because TinyUSB owns the native USB peripheral on GPIO19/20.

RFQ-EEG-001 Rev E E-28 asks for TP1-TP18 on the carrier plus exactly this 1x6 UART debug header at J26. The carrier provides both, so **there is no E-28 deviation**; the 2x5 1.27 mm JTAG/SWD header described in earlier drafts is withdrawn and no JTAG connector is fitted.

The DevKitC's two USB-C connectors are on the **non-isolated** side of the ADuM4160 barrier. They are used at the factory and on the bench only; the finished kit exposes only the isolator module's host USB-C (E-24, S-03), and T18 checks that the enclosure does not give a participant access to them.

### 2.4 `sdkconfig.defaults`, in full

This is the shipped file, verbatim. Every line is a requirement, not a preference.

*Corrected 2026-09-02.* The block below had stopped being verbatim. The five lines this section used to list under "not yet in the file and must be added before the first build" **are in the file now** – they were added on 2 September when the first real build refused to

proceed without them – and one of the five was named wrongly: there is no `CONFIG_TINYUSB_VENDOR_ENABLED` in `esp_tinyusb` 1.4.x, and ESP-IDF warns rather than fails on a key it does not recognise, so the setting looked present while `CFG_TUD_VENDOR` stayed 0 and the WebUSB interface did not exist. The key is `CONFIG_TINYUSB_VENDOR_COUNT`. The file is re-quoted here as it stands, with its own commentary, because a section headed “in full” that is not in full is worse than no quotation at all.

```
# FW-EEG-001 -- build configuration for EEG-CAR-01 Rev B.
# Every line here is a requirement, not a preference.  Changing one
changes the instrument.

CONFIG_IDF_TARGET="esp32s3"

# ---- flash and PSRAM
-------------------------------------------------------------
# The -N16R8 module has 16 MB quad flash and 8 MB OCTAL PSRAM.  The
octal PSRAM uses
# GPIO35, 36 and 37, which is why those three pins are not connected on
the carrier
# (ECO-EEG-009).
CONFIG_ESPTOOLPY_FLASHSIZE_16MB=y
CONFIG_ESPTOOLPY_FLASHMODE_QIO=y
CONFIG_ESPTOOLPY_FLASHFREQ_80M=y
CONFIG_SPIRAM=y
CONFIG_SPIRAM_MODE_OCT=y
CONFIG_SPIRAM_TYPE_AUTO=y
CONFIG_SPIRAM_SPEED_80M=y
CONFIG_SPIRAM_USE_MALLOC=y
CONFIG_SPIRAM_MALLOC_ALWAYSINTERNAL=4096

# ---- partitions
------------------------------------------------------------------
CONFIG_PARTITION_TABLE_CUSTOM=y
CONFIG_PARTITION_TABLE_CUSTOM_FILENAME="partitions.csv"
CONFIG_BOOTLOADER_APP_ROLLBACK_ENABLE=y
CONFIG_BOOTLOADER_APP_ANTI_ROLLBACK=y

# ---- USB
---------------------------------------------------------------------
----
# GPIO19 and GPIO20 are the USB PHY.  Using TinyUSB in device mode
takes the PHY away
# from the USB-Serial-JTAG console, so UART0 on GPIO43/44 is the only
console -- which
# is why J26 exists.
CONFIG_TINYUSB_CDC_ENABLED=y
CONFIG_TINYUSB_CDC_COUNT=1
CONFIG_TINYUSB_DESC_USE_ESPRESSIF_VID=n
CONFIG_ESP_CONSOLE_UART_DEFAULT=y
CONFIG_ESP_CONSOLE_UART_NUM=0

# ---- radio: never initialised (RFQ E-18)
-----------------------------------------
CONFIG_ESP_WIFI_ENABLED=n
CONFIG_BT_ENABLED=n
```

```
# ---- security (RFQ F-19)
# ------------------------------------------------------------
# Enabled for PHASE 2 ONWARD production builds only.  The Phase 1
prototypes are built
# with the sdkconfig.phase1 overlay and burn no eFuses.  The signing
key never leaves the
# programme; the manufacturer flashes a pre-signed image.  See FW-
EEG-001 section 7.5.
CONFIG_SECURE_BOOT=y
CONFIG_SECURE_BOOT_V2_ENABLED=y
CONFIG_SECURE_SIGNED_APPS_RSA_SCHEME=y
CONFIG_SECURE_BOOT_BUILD_SIGNED_BINARIES=n
CONFIG_SECURE_FLASH_ENC_ENABLED=y
CONFIG_SECURE_FLASH_ENCRYPTION_MODE_RELEASE=y

# ---- timing
# ------------------------------------------------------------------------
# The DRDY interrupt is the only place the sample counter moves (RFQ
E-19), so it must
# be in IRAM and must not be delayed by a flash cache miss.
CONFIG_FREERTOS_HZ=1000
CONFIG_ESP_INT_WDT_TIMEOUT_MS=300
CONFIG_ESP_TASK_WDT_TIMEOUT_S=10
CONFIG_SPI_MASTER_ISR_IN_IRAM=y
CONFIG_GPIO_CTRL_FUNC_IN_IRAM=y

# ---- storage
# ------------------------------------------------------------------------
-
CONFIG_FATFS_LFN_HEAP=y
CONFIG_FATFS_MAX_LFN=255

# ------------------------------------------------------------
added to close FW-D11
# FW-EEG-001 section 2.4 headed these "not yet in the file and must be
added before the
# first build", and they were still absent.  Without the first three
the vendor interface
# and the custom descriptor set do not exist, so WebUSB does not
enumerate and the
# _Static_assert on the MS OS 2.0 descriptor length never gets a chance
to fire.
# CONFIG_TINYUSB_VENDOR_COUNT, not CONFIG_TINYUSB_VENDOR_ENABLED.
#
# esp_tinyusb 1.4.x has no option called TINYUSB_VENDOR_ENABLED, and
ESP-IDF does not fail
# on a key it does not recognise -- it warns and carries on.  So the
setting looked present,
# CFG_TUD_VENDOR stayed 0, tusb.h declared none of the tud_vendor_*
functions, and the
# WebUSB interface this device is built around would simply not have
existed.  It surfaced
# as "implicit declaration of function 'tud_vendor_mounted'" the first
time the firmware was
# put in front of a real compiler.
```

```
CONFIG_TINYUSB_VENDOR_COUNT=1
CONFIG_TINYUSB_DESC_CUSTOM=y
CONFIG_TINYUSB_TASK_PRIORITY=18
CONFIG_BOOTLOADER_APP_SECURE_VERSION=1
CONFIG_BOOTLOADER_APP_ANTI_ROLLBACK=y
CONFIG_ESP_TASK_WDT_EN=y
```

`CONFIG_SECURE_BOOT_BUILD_SIGNED_BINARIES=n` is deliberate. The build container never holds the signing key; signing is a separate `espsecure.py sign_data` step performed on the custodian's offline machine (section 7.5). `CONFIG_SECURE_SIGNED_APPS_RSA_SCHEME` selects RSA-3072 for secure boot v2. That is independent of the ATECC608B's P-256 device key: one authenticates firmware to the chip, the other authenticates recordings to the programme.

**The five lines this section used to call missing are now present**, and this is what became of each. *Corrected 2026-09-02: the table below used to be headed "lines that are not yet in the file and must be added before the first build".*

| Line | Status | Note |
|---|---|---|
| `CONFIG_TINYUSB_VENDOR_COUNT=1` | **added** | FW-D03. This row used to name `CONFIG_TINYUSB_VENDOR_ENABLED`, which is not an option that exists; see above |
| `CONFIG_TINYUSB_DESC_CUSTOM=y` | **added** | FW-D02. It supplies the descriptors as data; the three callbacks `main.c` also defined had to be **deleted**, not kept |
| `CONFIG_TINYUSB_TASK_PRIORITY=18` | **added** | sits below `sample_task` at 23, as required |
| `CONFIG_BOOTLOADER_APP_SECURE_VERSION=1` | **added** | `CONFIG_BOOTLOADER_APP_ANTI_ROLLBACK=y` is meaningless without it |
| `CONFIG_ESP_TASK_WDT_EN=y` | **added, and the subscription is not written** | The option is on. Nothing subscribes `sample_task` or the SD path to the watchdog, so a stalled SD write still does not reset the unit. That half is open and is carried in section 10 |

**Phase 1 overlay.** The five security lines above are off for the two Phase 1 prototypes, which run **unsigned images and burn no eFuses at all**, so the firmware volunteer can re-flash them at the bench. Build Phase 1 images with `idf.py -DSDKCONFIG_DEFAULTS="sdkconfig.defaults;sdkconfig.phase1" build`, where `sdkconfig.phase1` sets `CONFIG_SECURE_BOOT=n`, `CONFIG_SECURE_FLASH_ENC_ENABLED=n` and `CONFIG_BOOTLOADER_APP_ANTI_ROLLBACK=n`. **That file is now in the package**, as `firmware/sdkconfig.phase1`, and is listed in full below. Secure boot and flash encryption are enabled from Phase 2 onward; see section 7.5.

The overlay is the whole of the phase switch. It turns security options off and adds none, so a Phase 1 image and a Phase 2 image are the same firmware built two ways, and a Phase 1 prototype can be re-flashed at the bench for ever. `sdkconfig.defaults` is never edited to move between phases – editing it is how a prototype gets its eFuses burnt by accident.

**`sdkconfig.phase1`, in full:**

*Corrected 2026-09-02.* The quotation below had also stopped being verbatim, and the divergence was load-bearing rather than cosmetic. The file had acquired a second block of options appended after the ones quoted here, and one of them was `CONFIG_BOOTLOADER_APP_ANTI_ROLLBACK=y` – twenty lines below the deliberate `=n`. A later duplicate of the same key silently wins, so the Phase 1 intent was being undone by a line that read as an addition. It also made the build fail outright: ESP-IDF refuses anti-rollback on a partition table carrying a `factory` partition, and `partitions.csv` has one. That is how it was found – the first real ESP-IDF build stopped there. The file is re-quoted as it stands.

```
# FW-EEG-001 section 2.4 -- Phase 1 overlay for the two prototypes.
#
# Use:  idf.py
-DSDKCONFIG_DEFAULTS="sdkconfig.defaults;sdkconfig.phase1" build
#
# Phase 1 prototypes run UNSIGNED images and BURN NO eFUSES (RUL-
EEG-021 section B), so the
# firmware volunteer can re-flash them over UART indefinitely.  Do not
put these lines in
# sdkconfig.defaults and do not build a Phase 2 unit with this overlay:
a secure-boot-v2
# bootloader burns its eFuses on first boot, and with
# CONFIG_SECURE_FLASH_ENCRYPTION_MODE_RELEASE=y that unit can never be
re-flashed over UART
# again.  Phase 2 onward builds from sdkconfig.defaults alone; see
section 7.5.

CONFIG_SECURE_BOOT=n
CONFIG_SECURE_BOOT_V2_ENABLED=n
CONFIG_SECURE_SIGNED_APPS_RSA_SCHEME=n
CONFIG_SECURE_BOOT_BUILD_SIGNED_BINARIES=n
CONFIG_SECURE_FLASH_ENC_ENABLED=n
CONFIG_SECURE_FLASH_ENCRYPTION_MODE_RELEASE=n

# Anti-rollback is an eFuse mechanism.  With no eFuses burnt there is
no anti-rollback and
# a downgrade is possible on a Phase 1 prototype; that is accepted and
stated in section 7.5.
CONFIG_BOOTLOADER_APP_ANTI_ROLLBACK=n

# ---------------------------------------------------------------
added to close FW-D11
# FW-EEG-001 section 2.4 headed these "not yet in the file and must be
added before the
# first build", and they were still absent.  Without the first three
the vendor interface
# and the custom descriptor set do not exist, so WebUSB does not
enumerate and the
```

```
# _Static_assert on the MS OS 2.0 descriptor length never gets a chance
to fire.
# CONFIG_TINYUSB_VENDOR_COUNT, not CONFIG_TINYUSB_VENDOR_ENABLED.
#
# esp_tinyusb 1.4.x has no option called TINYUSB_VENDOR_ENABLED, and
ESP-IDF does not fail
# on a key it does not recognise -- it warns and carries on.  So the
setting looked present,
# CFG_TUD_VENDOR stayed 0, tusb.h declared none of the tud_vendor_*
functions, and the
# WebUSB interface this device is built around would simply not have
existed.  It surfaced
# as "implicit declaration of function 'tud_vendor_mounted'" the first
time the firmware was
# put in front of a real compiler.
CONFIG_TINYUSB_VENDOR_COUNT=1
CONFIG_TINYUSB_DESC_CUSTOM=y
CONFIG_TINYUSB_TASK_PRIORITY=18
CONFIG_BOOTLOADER_APP_SECURE_VERSION=1
# NOT CONFIG_BOOTLOADER_APP_ANTI_ROLLBACK=y here.  It was appended in
this block and it
# contradicted the deliberate `=n` twenty lines above, which explains
why a Phase 1
# prototype has no anti-rollback: it is an eFuse mechanism and no
eFuses are burnt.  A
# later duplicate of the same key silently wins, so the Phase 1 intent
was being undone by
# a line that looked like an addition.  It also makes the build fail
outright -- ESP-IDF
# refuses anti-rollback on a table that has a `factory` partition, and
partitions.csv has
# one -- which is how it was found: the first real ESP-IDF build
stopped here.
CONFIG_ESP_TASK_WDT_EN=y

# ------------------------------------------------------------------ IRAM
headroom
# The first real build came back with static IRAM at 100.0 % -- 16,383
of 16,384 bytes,
# ONE byte free.  That is a cliff, not a pass: the next function anyone
marks IRAM_ATTR
# fails the link with an error naming a section rather than a cause,
and this design has
# more interrupt work coming (the E-13 tone scheduler, the E-12 onset
detector).
#
# Two ISRs are kept in IRAM by default and neither is used by this
firmware.  There is no
# SPI slave anywhere in the design -- the ESP32-S3 is the master and
both ADS1299 modules
# are slaves -- and nothing uses gptimer; `grep spi_slave` and `grep
gptimer` over main/
# return nothing.  Reclaiming them costs this design nothing.
#
# SPI MASTER stays in IRAM deliberately: sample_task() reads the
converters on every DRDY
```

```
# and that path must survive a flash-cache stall.
CONFIG_SPI_SLAVE_ISR_IN_IRAM=n
CONFIG_GPTIMER_ISR_HANDLER_IN_IRAM=n
#
# MEASURED AFTERWARDS, AND IT DID NOT HELP.  The figure is still 16,383
of 16,384 with one
# byte free, byte for byte, so whatever fills that pool it is not these
two ISRs.  The
# change is kept because carrying interrupt handlers for a bus and a
timer this firmware
# does not use is wrong either way, but it is NOT the fix and must not
be mistaken for one.
# Either the pool is genuinely full and a bring-up engineer has to
choose what leaves IRAM,
# or esp_idf_size is reporting against a fixed 16 KB window that is not
the real limit on
# an ESP32-S3 with octal SPIRAM and XIP.  Deciding which needs the
linker map read against
# hardware, so it is carried as an open item rather than guessed at
from here.
```

The three security lines named in the paragraph above are the ones that matter; the other four are their dependants, set explicitly so that `idf.py` reports the same configuration on every bench rather than resolving them silently.

Two blocks follow them and neither is a security line. The first repeats the FW-D02 / FW-D03 / FW-D11 options of the previous subsection; an overlay build reads `sdkconfig.defaults` first and then this file, so those four lines are **redundant rather than load-bearing** – they are kept because a reader of this overlay alone should be able to see the whole of what a Phase 1 image is built with. The second is the pair of IRAM lines, `CONFIG_SPI_SLAVE_ISR_IN_IRAM=n` and `CONFIG_GPTIMER_ISR_HANDLER_IN_IRAM=n`: an attempt at the open item of section 9 that **did not work**, is kept for a different reason than the one it was made for, and says so in the file.

**`sdkconfig.qemu` – an emulator overlay, and not a configuration for any unit.** Added 2 September 2026 so the firmware could be run without hardware (section 8.2). It differs from the shipped build in exactly one option, and that option is why the QEMU boot gets as far as it does:

```
# FW-EEG-001 -- overlay for running the firmware under QEMU.
#
# NOT a build configuration for any real unit.  It exists so the
firmware can be exercised
# without hardware, and it differs from the shipped build in exactly
one way that matters.
#
# QEMU's esp32s3 machine does not emulate octal PSRAM.  The shipped
configuration aborts
# when PSRAM is absent -- "Failed to init external RAM!" -- and the
unit then reboots in a
# loop.  That is defensible on real hardware, because the 6 MiB ring
buffer of F-06 lives
# in PSRAM and a unit without it cannot do its job; it is also, on a
field device, a
# failure mode that says nothing to anyone. A unit that boot-loops in a
```

```
participant's home
# cannot be told apart from a dead battery or a bad cable, and the
browser tool cannot ask
# it what is wrong because it never gets far enough to enumerate. See
KNOWN_ISSUES.
CONFIG_SPIRAM_IGNORE_NOTFOUND=y
```

Never build a unit with this overlay. `CONFIG_SPIRAM_IGNORE_NOTFOUND=y` turns a missing 6 MiB ring buffer – which is F-06, and the whole of the retransmit story – from a refusal to boot into a silent degradation, and a field unit that records without a ring is a unit that loses data the host asked it to keep.

### 2.5 `partitions.csv`, in full

```
# FW-EEG-001 partition table -- A/B OTA with rollback (RFQ F-20), 16 MB
flash.
# Name,     Type, SubType,  Offset,   Size,     Flags
nvs,        data, nvs,      0x9000,   0x6000,
otadata,    data, ota,      0xf000,   0x2000,
phy_init,   data, phy,      0x11000,  0x1000,
factory,    app,  factory,  0x20000,  0x300000,
ota_0,      app,  ota_0,    0x320000, 0x300000,
ota_1,      app,  ota_1,    0x620000, 0x300000,
calib,      data, nvs,      0x920000, 0x8000,
prov,       data, nvs,      0x928000, 0x8000,
storage,    data, fat,      0x930000, 0x6D0000,
```

| Partition | Size | Contents |
|---|---|---|
| `nvs` | 24 kB | general non-volatile settings |
| `otadata` | 8 kB | two 4 kB sectors selecting the boot slot |
| `phy_init` | 4 kB | present but unused; the radio is never initialised (E-18, S-06) |
| `factory` | 3 MB | the recovery image flashed at end of line and never overwritten in the field |
| `ota_0`, `ota_1` | 3 MB each | the A/B pair of F-20 |
| `calib` | 32 kB, NVS namespace `eegcal` | the constants written at T6 from T7, T10, T12 and T17 |
| `prov` | 32 kB, NVS namespace `eegcfg` | USB PID, hardware revision, unit serial, ATECC serial, provisioning timestamp |

*Corrected 2026-09-02.* This table was right and the firmware did not implement it. `drivers.c` opened namespace `"tiov"` in the **default** `nvs` partition for all four keys, so the two partitions above were allocated at 0x920000 and 0x928000 and never written – 64 kB reserved and unused – while provisioning and calibration went into the default partition instead. That partition is `0x6000` = **24 kB** and is shared with system state, so the 32 kB calibration set this table specifies could not have fitted in it: a full-size calibration write would have failed with an out-of-space error naming a partition no

document said was involved. `drv_nvs_set_str()`, `drv_nvs_set_blob()` and `unit_serial_into()` now open `calib/eegcal` for the calibration blob and `prov/eegcfg` for identity, initialising each partition on first use and erasing only that partition if it has never been written.

**Reading the calibration back is now possible.** `CMD_READ_CALIBRATION` (**0x4A**) returns a slice of the stored blob – payload `offset_lo offset_hi length` – so a host can compare byte for byte with what it sent. TST-EEG-004 T6's acceptance limit is that the unit reads back what was written, and until this opcode existed nothing could read it back at all. It is chunked because a full calibration set does not fit in one frame, and it is exempt from the provisioning-mode gate because T6 runs after provisioning, on a unit that has left that mode and may have had its configuration zone locked. | `storage` | 6.81 MB, FAT | block audio (F-11) and the 1 kHz reference tone (E-16, F-21) |

Arithmetic check: `0x930000 + 0x6D0000 = 0x1000000` = 16 MB exactly, so the table fills the device with no overlap.

**Measured, 2 September 2026.** *This paragraph used to read "a reference application build is expected at roughly 1.1 MB … **Calculated, not measured** – nothing has been built."* It has been built. `idf.py size` reports a **405,245-byte** image; `eeg_field_kit.bin` in `firmware/release/` is **405,360 bytes** on disk, the difference being the padding and the appended SHA-256 that `esptool` writes into the flashable file. Against a 3 MB slot that is **13 %** used, so the headroom for the FAT, I2S and SDMMC code the drivers still need is large. The QEMU boot log shows the bootloader loading exactly this table and taking the app from `factory` at `0x20000`, so the offsets in it are not merely arithmetic any more.

The one memory figure that is **not** comfortable is static IRAM: 16,383 of 16,384 bytes used, one byte free. That is section 10 item 17 and it is open.

At 48 kHz, 16-bit mono, the 6.81 MB FAT partition holds about **71 seconds** of audio (calculated). If a block's stimulus audio is longer than that it must be served from the microSD card instead. That decision is open and is listed in section 10.

---

## 3. The pin map

`main/board_pins.h` is **generated** from `tools/design.py` so that the firmware and the board cannot disagree. Do not edit it by hand: edit `design.py` and regenerate.

**The GPIO map itself lives in ICD-EEG-006 section 5**, as the J6 and J7 way tables, and is not restated here. Read the GPIO number and the header way for any net there. What this document owns is the set of macro names the generated header defines, and the four constraints below that a firmware author has to know.

| Macro | Net | Macro | Net |
|---|---|---|---|
| `PIN_SR_LATCH` | LED_SR_LATCH | `PIN_I2S_BCLK` | I2S_BCLK |
| `PIN_I2C_SDA` | SDA | `PIN_USB_DN` | USB_DN |
| `PIN_I2C_SCL` | SCL | `PIN_USB_DP` | USB_DP |
| `PIN_ENV_CMP` | ENV_CMP | `PIN_MIC_MUTE` | MIC_MUTE |

| Macro | Net | Macro | Net |
|---|---|---|---|
| PIN_BTN_A | BTN_A | PIN_SD_CMD | SD_CMD |
| PIN_BTN_B | BTN_B | PIN_SD_CLK | SD_CLK |
| PIN_BTN_STOP | BTN_STOP | PIN_SD_D0 | SD_D0 |
| PIN_I2S_DIN | I2S_DIN | PIN_SR_DATA | LED_SR_DATA |
| PIN_I2S_LRCK | I2S_LRCK | PIN_SR_CLK | LED_SR_CLK |
| PIN_I2S_DOUT | I2S_DOUT | PIN_UART0_TX | UART_TX |
| PIN_ADS_CS | CS | PIN_UART0_RX | UART_RX |
| PIN_ADS_MOSI | MOSI | PIN_VBUS_DET | VBUS_DET |
| PIN_ADS_SCLK | SCLK | PIN_CHG_CE | CHG_CE |
| PIN_ADS_MISO | MISO | PIN_LED_V | LED_PWM |
| PIN_ADS_DRDY | DRDY | PIN_RESERVED_PSRAM_D5/D6/D7 | not connected |
| PIN_ADS_START | START | PIN_RESERVED_VDD_SPI | not connected |
| PIN_ADS_RESET | RESET | | |

**ECO-EEG-009.** The Rev A map put LED_SR_DATA, LED_SR_CLK and LED_SR_LATCH on GPIO35, 36 and 37. On the ESP32-S3-DevKitC-1-N16R8 those three pins carry the **octal PSRAM**; they are not available as user IO and the firmware pin map was unbuildable. Rev B moves the shift register to GPIO41, 42 and 0 and drops the microSD interface to one-bit SDMMC to free them. The sustained microSD write requirement of E-20 is about 70 kB/s – the 50.7 kB/s of frame payload calculated in section 5.8 plus STATUS and SIGNATURE frames and filesystem overhead – against about 2 MB/s available in one-bit mode, so the headroom is ample (calculated). GPIO45 is the VDD_SPI strapping pin and is left open; a pull-up on it would set VDD_SPI to 1.8 V and the module would not boot.

main.c carried the Rev A defines until 2 September 2026, and a build of that source would have produced a unit whose contact lights did not work and whose PSRAM was corrupted by lights_write(). The block is deleted and main.c:48 includes the generated board_pins.h, so FW-D12 is closed; the aliases that follow the include exist only so the rest of the file keeps its original names. The generated header is the only pin map, and it comes from design.py: regenerate it, never edit it, and the firmware and the board cannot disagree.

board_pins.h also carries N_CH 16, N_DEV 2, ADS_GAIN_EEG 24, ADS_GAIN_EMG 12, ADS_GAIN_ENV 1, N_LIGHTS 8 and LIGHT_PHASE_HZ 240, and documents the dark-at-boot guarantee: LED_V is GPIO48, an input at reset, so no current can flow through any light whatever the shift register contains (E-27). LIGHT_PHASE_HZ is the **nominal** figure; the driver quantises it to the FreeRTOS tick and runs at about 250 Hz (section 3.2).

**Verified by** a build with -Werror plus T11 (contact-light colour in all three states) and T3 (board boots and draws expected current). *Status 2026-09-02: the build half is done –*

ESP-IDF v5.2.5, clean at `-Wall -Werror=all`, with `board_pins.h` compiled into the released image. T11 and T3 both need a unit and neither has been run. **Signed off by** the firmware volunteer against the generated header, with the generation re-run recorded in the release manifest – that sign-off has not happened.

### 3.1 What the layout changed, and what it means for the firmware

Two things changed in the carrier during layout, and the firmware author needs to know both even though neither moves a pin.

**The carrier is 150.0 x 130.0 mm, not the 130 x 124 mm of package v1.** Thirty connectors, 211 parts and 156 nets would not close at the smaller size.

**The carrier is a four-layer board: L1 signal, L2 reference plane, L3 reference plane, L4 signal.** Package v1 asserted that two layers would be enough and cheap to route. Actually doing the layout showed that it is not: on two layers the bottom side has to be both the reference plane and the second routing surface, and it cannot be both. Four layers give two full routing surfaces and a continuous reference under every analogue trace, which is what DSN-EEG-002 section 13's "layout rules that are requirements, not preferences" ask for and a swiss-cheesed two-layer pour cannot deliver. Both inner layers carry AGND_REF left of x = 62 mm and DGND right of it, tied at the R90 star point only, which is the zoning and star-point rule of DSN-EEG-003 section 3.3; the isolation keep-out of that same section – x >= 141 mm, y = 2 to 22 mm, no copper on any layer – is where the harness barrier crosses the board. Vias are through vias only, 0.60 mm pad on a 0.30 mm finished hole; there are no blind, buried, back-drilled, filled or plugged vias. The stack-up is mask / 35 um L1 / prepreg 0.200 / 17 um L2 / core 1.065 / 17 um L3 / prepreg 0.200 / 35 um L4 / mask, 1.60 mm +/- 10 % finished. The enclosure grew with the board: POD-P1 base 163.0 x 143.0 x 58.0 mm external and 158.0 x 138.0 x 55.5 mm internal, MP-01 module plate 146.0 x 126.0 x 3.0 mm.

**The routing closed, and the board it closed on is withdrawn.** The firmware author will be handed a board file that will change, and should know why. An external layout engineer read the Rev B board between 3 and 5 September 2026, declined the review and advised a redesign; **Rev B is withdrawn from fabrication** (ECO-EEG-016 section 2, ECO-EEG-030) and **Rev C is unrouted**, with its placement and routing bought from an external layout desk. What follows is the Rev B result, kept because it is the measured state of the data in the tree. `kicad/EEG-CAR-01_RevB_DRC_report.txt` is the authority. It records 3 745 track segments and 552 through vias on the four layers, each reference plane one continuous island per net, a smallest measured clearance of 0.260 mm on L1, 0.285 mm on the two planes and 0.275 mm on L4 against a 0.20 mm rule, a narrowest conductor of 0.20 mm, a smallest plated hole of 0.30 mm, all 145 nets one connected copper island with none left without copper, no digital net anywhere in the analogue zone, exactly one AGND_REF-to-DGND bridge and exactly one HARN_SHIELD-to-DGND bridge, and the isolation strip clear of copper on all four layers – that last one because the report says so, not because it was assumed. It records **zero violations** – no clearance, width, annular-ring, hole-size, edge, non-plated-hole, isolation or via keep-out violation, and no unclosed connection. It also records that **169 connections were routed at relaxed geometry**: 36 took a conductor narrower than the 0.25 mm preferred width and 133 kept full width but took a reduced gap, all of them at or above the 0.20 mm minimum. **None of that data is released for fabrication or for review**: Rev B is withdrawn and Rev C has no routing yet. **Nothing in Rev C changes a pin assignment**, so `board_pins.h` and every table in this document stand as written; if a later change to Rev C does move a pin, it is its own ECO and it obliges a regeneration and a diff of the header before the next build.

The firmware consequence is narrow and it is procedural: `board_pins.h` is generated from `design.py`, so every geometry change obliges a regeneration and a diff of the header against the previous release before the next build, and the release manifest records that the regeneration was run. The board field in the provisioning record (section 7.4) stays `EEG-CAR-01 Rev B`, which is the electrical revision; the geometry change is carried as ECOs against it and does not bump `bcdDevice`.

### 3.2 The contact-light phase scheme, now written

Each of the eight sites carries a two-lead bicolour LED between its shift-register output Qn and the LED_V common on GPIO48, with a 1 kOhm series resistor R70-R77. With Vf = 2.0 V the per-site current is (3.3 - 2.0)/1000 = **1.3 mA**, and all eight together draw **10.4 mA** from GPIO48. Phase A (LED_V high, Qn low) shows green, phase B (LED_V low, Qn high) shows red, and alternating the two phases faster than the eye can follow shows amber. The requirement is that the alternation runs above 100 Hz; the fitted value is `LIGHT_PHASE_HZ 240`.

*Corrected 2026-09-02.* This subsection used to be headed “and the fact that it is not written”, and said “**None of this is implemented** … TST-EEG-004 T11 … cannot pass until the phase driver is written”. **It is written now**, in `lights_phase()` and `lights_task()` at `main.c:325-360`, and FW-D16 is closed. **E-27 is met in the design and in the source.** It has never been run: no unit exists, and QEMU has no shift register.

Three things about the implementation belong here because they are not obvious from the scheme above.

**The colour comes from the converter’s lead-off comparator, read at two thresholds.** A site that trips neither is **green**, a site that trips only the sensitive threshold is **amber** – marginal, re-gel it – and a site that trips both is **red**. That is the three-state indication E-27 asks for, taken from the measurement E-27 names, rather than a colour chosen by firmware state. The insensitive set is a subset of the sensitive one by construction, so the three states are exhaustive and cannot overlap.

*Corrected again 2026-09-02 (FW-D17).* This paragraph read “**The colour comes from both halves of the converter’s lead-off measurement.** The driver reads `LOFF_STATP` and `LOFF_STATN`, not one of them.” It did, and that was the defect. `ads_init()` enabled `LOFF_SENSP` only, so `LOFF_STATN` was zero on every channel for ever, the red term `p & n` was always zero, and **red was unreachable** – every site that had lost contact showed amber. Enabling `LOFF_SENSN` would not have fixed it: this board’s montage is **single-ended**, J2 carrying IN1 to IN8 with one shared `SRB1` and `BIASOUT`, and with `SRB1` closed all eight N bits report that one shared reference rather than eight sites. Sweeping `COMP_TH` is what the hardware actually supports. The two threshold values are the ADS1299’s documented endpoints, not measured trip points, so **the impedance at which a site turns amber, and the impedance at which it turns red, are not yet established** – TST-EEG-004 T11 sets them at first bring-up.

**The alternation is about 250 Hz, not 240.** `LIGHT_PHASE_HZ` is 240, so a half-phase is 2.083 ms; the task delays in FreeRTOS ticks and `CONFIG_FREERTOS_HZ` is 1000, so the half-phase quantises to 2 ms and the alternation runs at **250 Hz**. This is stated rather than hidden. The requirement is “above 100 Hz” and 250 Hz meets it with room; both half-phases quantise identically so the duty stays 50/50 and the colour does not shift; and T11 reads the red/green ratio with a colorimeter, which does not care about 4 % of frequency. A build that needs exactly 240 Hz needs a hardware timer rather than a task delay, and that is different work.

**Dark is dark by construction.** While a block is recording, or when the host has switched the lights off, `lights_task()` clears the shift register **and returns LED_V to an input** – the state it holds at reset. With the common floating no current can flow through any site whatever the register contains. Driving LED_V low would also be dark, but only for as long as the level is right.

**`CMD_LIGHTS` still does not do what section 6.3's argument column describes**, and this paragraph is corrected too: it used to say the source "answers `0x0B`, not implemented, to all three" of modes 2, 3 and 4. It does not. `handle_command()` takes the mode byte as a plain enable – `g_lights_enabled = c[1]` – and answers `0x00` OK to any value, while the colour is decided by the lead-off measurement rather than by the mode. So a host asking for "force red" gets a success and the automatic colour, which is a worse answer than a refusal. Forcing a colour for T11 is not implemented; the honest reply would be `0x0B` and the source does not send it. That is carried in section 10.

---

## 4. The USB device model

### 4.1 Composite configuration

| Item | Value |
|---|---|
| `bcdUSB` | `0x0210` – required for a BOS descriptor to be read |
| Device class | `TUSB_CLASS_MISC` / `MISC_SUBCLASS_COMMON` / `MISC_PROTOCOL_IAD` |
| Speed | full speed, 64-byte packets |
| Interface 0, 1 | CDC-ACM: notification EP `0x81` (8 B), data OUT `0x02`, data IN `0x82` (F-01, WebSerial path) |
| Interface 2 | vendor-specific bulk: OUT `0x03`, IN `0x83`, 64 B (F-01, WebUSB path) |
| `bMaxPower` | 100 mA declared; the instrument is battery powered and draws nothing from the host |
| Strings | 1 "TI One Voice", 2 "EEG field kit", 3 iSerial, 4 "EEG CDC", 5 "EEG WebUSB" |

Both interfaces carry the **identical** frame stream. Whichever the host opens first wins; if both are open, `frame_emit()` writes to both.

### 4.2 BOS, WebUSB, Microsoft OS 2.0

The BOS descriptor declares two platform capabilities.

| Capability | Vendor request | Serves |
|---|---|---|
| WebUSB platform | `bRequest = 1` | landing URL `https://one.witysk.org/eeg`, scheme byte 1 = https |
| Microsoft OS 2.0 platform | `bRequest = 2`, `wIndex = 7` | a 178-byte (`0xB2`) descriptor set |

The MS OS 2.0 set assigns the compatible ID `WINUSB` to the vendor interface and sets the registry property `DeviceInterfaceGUIDs`, so Windows binds WinUSB with no user action (F-02). The length is asserted at compile time: `_Static_assert(sizeof(ms_os_20_desc) == MS_OS_20_DESC_LEN, ...)` at `main.c:705`. *Corrected 2026-09-02: this read "That assert has never been evaluated (FW-D11); it is the test, and it fires on the first build."* The first build happened, the assert was evaluated against the resolved `esp_tinyusb` 1.4.5 macros, and it **held**. FW-D11 is closed. What that settles is the descriptor's length, not whether Windows binds WinUSB to it; that is T5.

**The GUID in the shipped source, `{8FE6D4D7-49DD-41E7-9486-49AFC6BFE475}`, is the TinyUSB example GUID.** It must be replaced with a GUID minted once for this programme and recorded in the release manifest, otherwise Windows may bind any other TinyUSB sample device on the same PC to the same interface class.

### 4.3 VID and PID

| Where | Value | Status |
|---|---|---|
| `main.c:673-674` | `0x1209` / `0x0000` | `0x0000` is not a valid product ID. *Line reference corrected 2026-09-02* |
| `provision.py DEFAULT_PID` | `0x0001` | pid.codes reserves 1209:0001 for testing only |
| Allocated | pending | pid.codes application under VID `0x1209` |

F-03 and F-18 say the manufacturer programs the assigned VID and PID at end of line, but `main.c` compiles them into a `static const tusb_desc_device_t`, so nothing can be programmed. **Decision, Rev B, unchanged:** `idProduct`, `bcdDevice` and `iSerialNumber` are read from NVS namespace `eegcfg` at boot, **before** `tinyusb_driver_install()`, with compiled-in fallbacks. A unit that boots on the fallback must enumerate as `iProduct` = "EEG field kit UNPROVISIONED" so that T5 fails loudly rather than shipping. `bcdDevice` = `0x0100` corresponds to hardware revision B.

The two Phase 1 prototypes may carry 1209:0001 with a written note that they are never given to a participant. Browser authorisation is keyed to VID, PID and the serial string, so the permission grant has to be re-made once the real PID lands. That is acceptable for two prototypes and unacceptable for a fleet.

---

## 5. Frame format as an implementation contract

RFQ-EEG-001 section 5.2 gives the field list. This section gives the bytes. All multi-byte integers are **little-endian** unless stated. Every frame crosses every boundary unchanged: microSD, USB and the browser-to-server websocket. One parser, one test suite.

### 5.1 Envelope

| Offset | Size | Field | Notes |
|---|---|---|---|
| 0 | 1 | `version` | 1 for this document. A host that sees another value must reject the frame, not parse it |
| 1 | 1 | `type` | 1 DATA, 2 STATUS, 3 EVENT, 4 GAP, 5 SIGNATURE, 6 CMD_ACK, 16 CMD (host to device) |
| 2 | 2 | `seq` | u16, increments per frame, wraps at 0xFFFF |
| 4 | 4 | `first_sample` | u32, the DRDY counter of the first sample. **The only timeline in the system** (E-19) |
| 8 | 1 | `rate_code` | 0 = 250 Hz, 1 = 500 Hz, 2 = 1000 Hz |
| 9 | 1 | `n_samples` | 20 at 1000 Hz, 10 at 500, 5 at 250; 0 for non-DATA frames |
| 10 | n | payload | per type, below |
| 10+n | 4 | `crc32` | IEEE 802.3 over bytes 0 to 10+n-1 |

The whole of the above is then COBS-encoded and a single `0x00` delimiter is appended. A decoder joining mid-stream resynchronises at the next `0x00`, which is what the delimiter is for. Header struct in Python: `struct.Struct("<BBHIBB")`, size 10.

### 5.2 DATA payload

`n_samples` records of exactly **50 bytes**:

| Offset in record | Size | Field |
|---|---|---|
| 0 | 48 | channels 1 to 16, each 24-bit two's complement, **big-endian**, in ADS1299 output order |
| 48 | 2 | auxiliary field, u16 little-endian |

Channel map, normative:

| Channel | Signal | Gain | Source |
|---|---|---|---|
| 1-8 | Fz, Cz, Pz, C3, C4, T7, T8, F7 | 24 | ADS module #1, J14 |

| Channel | Signal | Gain | Source |
|---|---|---|---|
| 9-11 | EMG cheek, submental, laryngeal | 12 | ADS module #2 ch1-3, J15-J17 |
| 12 | ENV_STIM | 1 | ADS module #2 ch4, J4.4 |
| 13 | ENV_VOICE | 1 | ADS module #2 ch5, J4.5 |
| 14 | ENV_ROOM | 1 | ADS module #2 ch6, J4.6 |
| 15-16 | spare / EOG | 24 | ADS module #2 ch7-8, J22 |

Channels 15 and 16 exist in the stream on every unit and are protected like every other electrode lead, but **the EOG panel sockets are not fitted in a standard build**; they are a Phase 2 option. The two channels record whatever J22 is left presenting.

**Channel 12 is the authoritative stimulus onset record** (F-11). Note that this numbering is the *stream* numbering and is not the same as the R1-R16 protection-network numbering in DSN-EEG-003, which is numbered by electrode lead and includes REF_L, REF_R and BIAS. The two lists are different things and must not be conflated.

A reversed daisy chain inverts the whole map. It is detected at T7 because a channel 9 reading gain 24 rather than 12 can only mean the modules are the other way round – see T27.

### 5.3 Auxiliary field

| Bit | Meaning | Source |
|---|---|---|
| 0 | BTN_A pressed | GPIO4, active low |
| 1 | BTN_B pressed | GPIO5 |
| 2 | BTN_STOP pressed | GPIO6 |
| 3 | stimulus comparator, **latched in the DRDY ISR** | GPIO3, E-12 |
| 4 | any lead-off asserted | LOFF_STATP |
| 5 | recording block in progress | firmware state |
| 6 | charger input present | GPIO46 VBUS_DET |
| 7 | microSD write healthy | SD driver |
| 8-11 | reserved, zero | |
| 12-15 | protocol reserved, zero | |

`main.c` sets bits 0 to 5 and samples bit 3 in the task rather than in the ISR (FW-D08).

Bit 3 reads GPIO3, the output of the U7 comparator. ECO-EEG-023, which is **open and not implemented**, would re-power U7 from DVDD3V3 and DGND and re-reference its inputs to a DVDD3V3/2 divider with the envelope AC-coupled into it, so that GPIO3 would swing a full 0 to 3.3 V. As released, U7 is on AVDD/AVSS and the clamp leaves 20 mV of logic margin. That is a change the safety and layout reviewer must check; the firmware side is unaffected beyond the improved margin.

### 5.4 STATUS payload – 24 bytes, once per second (F-09)

| Offset | Size | Field |
|---|---|---|
| 0 | 1 | battery percent, 0-100 |
| 1 | 1 | link type: 0 none, 1 CDC, 2 vendor |
| 2 | 2 | lead-off bits, channels 1-16 |
| 4 | 4 | microSD free space, MB |
| 8 | 4 | uptime, seconds |
| 12 | 2 | die temperature, i16, tenths of a degree C |
| 14 | 4 | clock-offset estimate, i32, microseconds. **Metadata only; never applied to the sample timeline** (F-16) |
| 18 | 2 | ring depth actually allocated, seconds – T15 records this rather than assuming it |
| 20 | 3 | firmware version, major / minor / patch |
| 23 | 1 | flags: b0 recording, b1 charging inhibited, b2 card present, b3 provisioned |

The die temperature at offset 12 is the ESP32-S3 internal sensor. It is **not** a battery temperature: there is no NTC net on the carrier and no thermistor way on J12 or J13, so the thermistor-monitored charging of RFQ S-04 is **not met and stays not met** as an open hardware item. The 45 degree C charge inhibit of RFQ E-23 is a property of the charger IC's own thermal regulation, not of anything the firmware reads.

### 5.5 GAP payload – 10 bytes (F-07)

| Offset | Size | Field |
|---|---|---|
| 0 | 4 | first sample index lost |
| 4 | 4 | last sample index lost |
| 8 | 2 | frames lost |

Silent loss is not permitted. `verify_stream.py` resumes its sample-index expectation at `last + 1` when it sees a GAP, which is the only way a legitimate loss is distinguished from a decoder fault. A GAP is also the marker the host uses to decide which range to recover from the microSD copy, which is now part of how F-06 is met – section 5.8.

### 5.6 SIGNATURE payload – 104 bytes, every 2048 samples (F-08)

| Offset | Size | Field |
|---|---|---|
| 0 | 64 | ECDSA P-256 signature, raw `r \|\| s`, 32 bytes each, big-endian |
| 64 | 4 | block index, u32 |
| 68 | 4 | first sample index of the block |
| 72 | 32 | block hash |

The chain, exactly as `verify_stream.py` computes it: `block_hash` is SHA-256 over the concatenation of every DATA frame body in the block **excluding its CRC**; the signed digest is `SHA-256(previous_chain_value || block_hash)`; the chain value becomes that digest. The signature is computed by the ATECC608B over the digest. Datasheet ATECC sign time is typically 50 to 70 ms; at 1000 Hz a block is 2.048 s, so the margin is large, but signing must not run on the sample task. Nothing has been measured on hardware.

### 5.7 EVENT payload

| Offset | Size | Field |
|---|---|---|
| 0 | 1 | event code: 1 block start, 2 block end, 3 button, 4 host mark |
| 1 | 1 | label length, 0 to 32 |
| 2 | n | label, UTF-8, maximum 32 bytes |

### 5.8 Bandwidth and ring depth – calculated

One frame at 1000 Hz is 10 header + 20 x 50 payload + 4 CRC = **1014 bytes**, which is 1015 bytes after COBS encoding and 1016 bytes on the wire once the delimiter is added. Fifty frames per second is therefore **50.7 kB/s of frame payload** – the figure every document in this package now uses – and about 50.8 kB/s on the wire.

E-20's "approximately 70 kB/s" and F-12's "approximately 64 kB/s" are not in conflict with that number and neither requirement changes. They are **allowances**: they include the once-per-second STATUS frames, the SIGNATURE frame every 2.048 s, and FAT filesystem overhead on the microSD copy, none of which is frame payload. 70 kB/s remains the microSD sizing number and the E-20 acceptance figure.

**Ring depth.** F-06 as originally written asks for at least three minutes of ring at 1000 Hz and sizes that at **12 MB**. The 12 MB is not a mistake and this document no longer calls it one: it is three minutes at E-20's 70 kB/s allowance, 180 x 70 kB = 12.6 MB rounded down, which is the figure DSN-EEG-003 section 5, TST-EEG-004 T15, RFQ-EEG-001 F-06 and the ECO-EEG-025 title all quote. Measured instead against the frame payload derived above, three minutes at 50.7 kB/s is **9.13 MB** (calculated). The two arithmetics differ because one counts the allowance and the other counts the payload, and the difference does not matter here, because the ESP32-S3-DevKitC-1-N16R8 (E-18, not substitutable) has **8 MiB** of PSRAM in total, before any heap. Neither 12 MB nor 9.13 MB can be allocated: `main.c` asked for 12 MiB, that allocation returned NULL, and every subsequent `xRingbufferSend()` dereferenced it. That was FW-D13 and it is

fixed – `RING_BYTES` is 6 MiB at `main.c:101`, and `app_main()` now aborts with a diagnostic naming the PSRAM configuration rather than carrying on if the allocation ever fails again.

**Ruling, carried as ECO-EEG-025.** `RING_BYTES = 6,291,456` bytes – a **6 MiB** ring, and the "6 MB ring" the rest of the package names.

| Figure | Value | How it is counted |
|---|---|---|
| Allocation | 6,291,456 bytes | `RING_BYTES`, 6 MiB |
| Depth at 1000 Hz, as the simulator reports it | **126 s** | `tools/simulate_production.py`, over the 50 bytes per sample the ring is sized to hold |
| Depth at 1000 Hz, over the complete framed stream | 124 s | 50.7 kB/s from the paragraph above, header, CRC and COBS included |
| Depth at 500 Hz, over the complete framed stream | 244 s | one frame every 20 ms carrying ten samples |
| Requirement, F-06 as relaxed by ECO-EEG-025 | **90 s** | plus unlimited backfill from the microSD copy |

`simulate_production.py` grades the check "F-06 as relaxed by ECO-EEG-025: at least 90 s of ring" and passes it at 126 s. Whichever of the two arithmetics is used the fitted ring is more than a third deeper than the relaxed requirement, so the choice between them changes no decision; both are given so that no reader has to guess which one produced a number quoted elsewhere.

**F-06 is relaxed to 90 seconds of ring at 1000 Hz plus unlimited backfill from the microSD copy.** The reason is worth stating: the ring exists to survive a USB stall or a browser tab losing focus, and the card holds the same byte stream in full, so anything older than the ring is recovered from the card rather than from PSRAM. Ninety seconds of live retransmit plus a complete card copy is a better answer than three minutes of PSRAM the module does not have. The device declares the depth it actually allocated in the STATUS frame and T15 grades against the declared value, not against any figure in the table above. The alternative – a 16 MB-PSRAM module – contradicts E-18 and is not recommended.

### 5.9 The microSD file layout

**This is the definition of what is on the card, and it is made once, here.** SVC-EEG-013 section 2 R2, TST-EEG-004 T14 and the programme's ingest tooling cite this section and do not restate it. The earlier SVC-EEG-013 layout – `/SESSIONS/<unit_serial>/<YYYYMMDD>T<HHMMSS>Z_<session_id>.eegs` with a 512-byte plain-text header – is withdrawn, for two reasons that are worth writing down. The unit has no real-time clock, so at the moment a file is created the device may not yet know the UTC time; it learns it from `CLOCK_XCHG`, which the host may send late or not at all, and a filename that cannot always be formed is not a filename rule. And an in-file header breaks the one contract section 5 exists to state: every frame crosses every boundary unchanged, so the card holds exactly the bytes that went to USB and one

parser reads both. The UTC start time the service engineer wants is kept, in the sidecar, where a missing clock exchange leaves a null instead of an unusable name.

| Item | Definition |
|---|---|
| Filesystem | **exFAT**, formatted by the programme before the card is issued. The firmware never formats a card |
| Volume label | `TIOV<nnnn>`, the four serial digits, so a card found loose can be returned to its unit |
| Directory | `/EEG/<unit_serial>/`, one directory per unit, `<unit_serial>` exactly the `TIOV-B-nnnn` string of section 7.2 |
| Session file | `<session_id>.eeg`, where `<session_id>` is the 16 bytes of `START_SESSION` rendered as **32 lowercase hexadecimal characters** |
| Session file content | The **byte-identical COBS frame stream** that went to USB: DATA, STATUS, EVENT, GAP and SIGNATURE frames, in the order emitted, each with its `0x00` delimiter. No in-file header, no index, no padding, no trailer |
| Sidecar | `<session_id>.json`, UTF-8, written at `STOP_SESSION` |
| Provisioning record | `/EEG/<unit_serial>/unit.json`, the section 7.4 identity block, written once at provisioning and never rewritten by a session |

The sidecar carries what a reader cannot recover cheaply from the stream, and nothing that contradicts it:

| Field | Type | Notes |
|---|---|---|
| `schema` | u8 | 1 for this document |
| `unit_serial` | string | as on the label |
| `session_id` | string | 32 hex characters, matching the filename |
| `board` | string | `EEG-CAR-01 Rev B` |
| `firmware_version` | string | and `image_sha256`, both as `GET_VERSION` reports them |
| `utc_start` | string or null | ISO 8601, derived from the last `CLOCK_XCHG` before the first frame. **Null if the host never sent one**, and a null is not an error |

| Field | Type | Notes |
|---|---|---|
| `rate_code`, `sample_rate_hz` | u8, u16 | as recorded |
| `first_sample`, `last_sample` | u32 | the DRDY counter at the ends of the file |
| `frames`, `gaps`, `signature_blocks` | u32 | counted as written |
| `public_key_fingerprint` | string | the section 7.4 form, so the card can be verified without the unit |
| `sha256` | string | over the `.eeg` file, written last |

Rules that follow from the above and are part of the definition. A session file whose sidecar is missing – power lost mid-session – **is still a valid recording**: the reader rebuilds every sidecar field except `utc_start` and `sha256` from the frames themselves, and records the sidecar as absent rather than the session as failed. A file is never appended to after `STOP_SESSION`, and a `<session_id>` is never reused. Card-full policy is **stop and flag in STATUS, never overwrite**; the free-space field of section 5.4 is what the host watches. At the 50.7 kB/s of section 5.8 a recording hour is **182.5 MB**, a three-hour session **548 MB** and a three-session loan about **1.10 GB** (calculated), so a 32 GB card holds roughly 175 recording hours and never fills within a loan.

Nothing above has been written by a real firmware. *Corrected 2026-09-02: this paragraph said `sd_append()` and `sd_free_mb()` "are stubs". They are not stubs any more – they are written, at `drivers.c:163` and `drivers.c:175` (section 1.2).* What has not changed is the part that matters: **no card has ever been written**, because no unit exists, and the SDMMC host did not initialise under QEMU because QEMU has no card slot. Nothing writes the sidecar. This is still the contract the driver must meet, not a description of an existing card.

---

## 6. The command channel

### 6.1 Host to device

A command is a normal frame with `type = 16` (CMD), `first_sample = 0`, `rate_code = 0`, `n_samples = payload length`. The payload is the opcode byte followed by its arguments. This is what `provision.py` and the browser tool both send.

`rx_poll()` now reads it that way (FW-D14, fixed 2 September 2026): after the CRC check it requires at least `FRAME_HDR_BYTES + 1` bytes, requires `out[0] == PROTO_VERSION` and `out[1] == FT_CMD`, and dispatches from `out + FRAME_HDR_BYTES`. A frame that fails any of those is dropped in silence, which is the right answer on a link where the alternative is acting on a frame the device did not understand.

Until that fix the whole decoded body went to `handle_command()`, which read byte 0 – the protocol version – as the opcode. `PROTO_VERSION` is 1 and `CMD_START_SESSION` is `0x01`, so every command from the host started a recording session, including the two that exist precisely so a host can talk to a unit without starting one. The lesson is in

section 8: a simulated device written from the same specification as the firmware shares the firmware's misreadings of it, and will not find this class of defect.

**Four images, not five, and the name has an underscore in it.** This command and ASM-EEG-007 section 6.1 step 4 used to disagree twice, and a factory can only follow one of them. Both are settled against the source rather than by preference:

- The image is `eeg_field_kit.bin`. `firmware/CMakeLists.txt` line 7 is `project(eeg_field_kit VERSION 0.2.0)` and ESP-IDF names the binary after the project, so ASM-EEG-007's `eeg_fieldkit.bin` was a file that would never exist. Corrected there.
- `0x930000 storage.fat` is REMOVED from this command. There is no `storage.fat` anywhere in the package to flash, and nothing in the firmware mounts the internal `storage` partition: the only FAT mount is `esp_vfs_fat_sdmmc_mount()` in `drivers.c`, which mounts the microSD CARD. The partition is allocated in `partitions.csv` and is currently unused. Leaving the line in would have stopped the flash step with a missing file, on every unit, at the factory.

### 6.2 Acknowledgement

Every command produces one CMD_ACK frame (`type = 6`) whose payload is:

| Offset in payload | Size | Field |
|---|---|---|
| 0 | 1 | opcode echoed |
| 1 | 1 | reserved, zero |
| 2 | 1 | status code |
| 3 | 1 | result length, bytes following |
| 4 | n | result bytes |

In absolute frame offsets that puts the status byte at 12 and the result at 14, which is what `provision.py` reads.

`main.c` sends this shape as of 2 September 2026. `ack_emit()` (`main.c:430-439`) is the single place a CMD_ACK is built for the general command channel, and the browser tool's own `parseAck()` decodes it in the interop harness of section 8. Before that, every ack went out as `{opcode, status}` with the status at offset 1 and no length, and IDENTIFY, LOOPBACK and CLOCK_XCHG skipped even that and returned their result at offset 0 with no opcode echo at all – so a host could not tell which command an acknowledgement answered and matched replies by arrival order.

**Every path uses it now.** *Corrected 2026-09-02: this paragraph read "One path still does not use it", and named `handle_provision()`.* The whole `0x40`-`0x4F` family returns through `ack_emit()` at `main.c:561`, including the refusal a command gets when it arrives outside provisioning mode, so FW-D20 is closed. It mattered: `provision.py` reads the status at absolute frame offset 12, which is payload offset 2, and under the old shape that was the first *result* byte – on `0x48` READ_PROVISION_STATE, the config-zone lock flag. An **unlocked** unit therefore reported 0 and the tool read it as success. Three of the interop harness's 57 checks now read a provisioning status at this offset, so

the shape cannot drift again without a test failing. End-of-line provisioning has still never been run against a real ATECC608B.

| Status | Meaning |
|---|---|
| `0x00` | OK |
| `0x01` | unknown opcode |
| `0x02` | bad length |
| `0x03` | argument out of range |
| `0x04` | wrong state (for example a block command outside a session) |
| `0x05` | interlock: VBUS_DET high, session refused (S-01) |
| `0x06` | not provisioned |
| `0x07` | hardware fault (bus NAK, converter not responding) |
| `0x08` | payload CRC or hash mismatch |
| `0x09` | locked: the ATECC configuration zone is already locked |
| `0x0A` | timeout |
| `0x0B` | not implemented in this build – the honest answer from a stub |

**The source uses `0x02` for two different things, and this table is only half of what a host sees.** *Recorded 2026-09-02.* `CMD_ENTER_PROV` (`0x40`) refuses when the ATECC608B configuration zone is already locked – which is the guard that stops a fielded unit being re-keyed over USB, and is intended – and it answers **`0x02`** for it, where this table assigns `0x02` to “bad length” and `0x09` to “locked”. `CMD_ATECC_WRITE_CONFIG` (`0x4B`), in the same handler, answers `0x09` for the same condition. So an operator reading this table alone, faced with a `0x02` from step 1, would go looking for a framing fault that is not there.

The code is the fact and the document does not get to overrule it, so it is written down rather than quietly renumbered: **`0x02` from `0x40` means “already provisioned, refused”, and `0x02` from anything else means bad length.** `provision.py` hints both readings at that step. The fix is one line in `main.c` – return `0x09` there, which is defined for exactly this and is what `0x4B` already returns – and it belongs to whoever owns `main.c`; it is carried in section 10 rather than pre-announced here as done.

`handle_provision()` also returns several codes this table does not define at all: `0x10` to `0x18` for individual driver failures (GenKey, pubkey read, serial read, each NVS write, the config write, the lock) and `0x20` for a short payload on `0x44` and `0x45`. They are distinguishable and they are useful at a station; they are not in section 6.2 and a host written to this table will render them as unknown. Also carried in section 10.

Any command that receives no acknowledgement within **2 s** is a failure; the host retries once and then reports. Long operations (`CMD_TIMING_SELFTEST`, `CMD_ATECC_GENKEY`) are given **20 s**.

### 6.3 Command table

| Opcode | Command | Arguments | Result | Errors | Test |
|---|---|---|---|---|---|
| `0x01` | START_SESSION | session_id 16 B | – | `0x04`, `0x05` if VBUS_DET high | T21 |
| `0x02` | STOP_SESSION | – | – | `0x04` | T21 |
| `0x03` | BLOCK_START | label_len u8, label <= 32 B | – | `0x02`, `0x04` | T13 |
| `0x04` | BLOCK_END | – | – | `0x04` | T13 |
| `0x05` | SET_RATE | rate_code u8 | – | `0x03` | T7, T14 |
| `0x06` | SET_GAIN | channel u8 1-16, gain_code u8 0-6 | – | `0x03` | T7 |
| `0x07` | IMPEDANCE | mode u8 (0 off, 1 AC 7.8 Hz, 2 AC 31.2 Hz), mask u16 | – | `0x03`, `0x07` | T10 |
| `0x08` | RETRANSMIT | seq_from u16, seq_to u16 | frames replayed u16 | `0x03` if outside the ring | T15 |
| `0x09` | TIMING_SELFTEST | n_tones u8 (40) | median x100 i16, p95 x100 i16, verdict u8, 40 residuals i16 | `0x0B` until the codec driver has been brought up on hardware; `main.c` returns a 3-byte result today, not the result column above | T13 |

| Opcode | Command | Arguments | Result | Errors | Test |
|---|---|---|---|---|---|
| `0x0A` | LIGHTS | mode u8 (0 off, 1 auto, 2 green, 3 red, 4 amber), optional mask u8 | – | `0x00` for modes 0 to 4; `0x0B` UNIMPLEMENTED for any other mode; `0x02` BAD_LENGTH if the mode byte is absent. *Corrected twice on 2026-09-02. First: the phase driver now exists (section 3.2), so mode 1 shows all three colours. Then **FW-D18**: this row said modes 2, 3 and 4 were not implemented and that the source wrongly answered `0x00` OK to them. **They are implemented now** – the mask in `c[2]`, when present, selects the sites the forced colour applies to, and defaults to all eight. A host that asks for a mode the firmware does not have gets a refusal rather than a false success* | T11 |

| Opcode | Command | Arguments | Result | Errors | Test |
|---|---|---|---|---|---|
| `0x0B` | CLOCK_XCHG | host time i64 us | sample_index u32, device time i64 us | – | – |
| `0x0C` | PLAY_AT | asset_id u8, start_sample u32, level i16 tenths dB | – | `0x04`, `0x0B` | T12b, T13 |
| `0x0D` | FW_UPDATE_BEGIN | image_len u32, sha256 32 B | slot u8 | `0x04` during a session | T25 |
| `0x0E` | FW_UPDATE_DATA | offset u32, bytes <= 512 | bytes accepted u16 | `0x08` | T25 |
| `0x0F` | IDENTIFY | – | proto version u8, fw major u8, fw minor u8, board revision letter u8, ring bytes u32, capability flags u32, current rate code u8, supported rate count u8, unit serial NUL-terminated | – | TOOL-EEG-022 step D1 |
| `0x10` | LOOPBACK | up to 240 bytes | the same bytes, unchanged | none: the source truncates a longer payload to 240 bytes rather than refusing it | TOOL-EEG-022 steps D4-D6 |
| `0x11` | SET_HP_LEVEL | level i16 tenths dB | level read back from the codec | `0x03`, `0x0B` | T17 |
| `0x12` | REF_TONE | freq u16 Hz, level i16, duration u16 ms | env_voice u16, env_room u16 | `0x0B` | T17, E-16 |
| `0x13` | LOAD_AUDIO_BEGIN | asset_id u8, len u32, sha256 32 B | – | `0x03` | T12b |

| Opcode | Command | Arguments | Result | Errors | Test |
|---|---|---|---|---|---|
| 0x14 | LOAD_AUDIO_DATA | offset u32, bytes <= 512 | bytes accepted u16 | 0x08 | T12b |
| 0x15 | LOAD_AUDIO_END | sha256 32 B | – | 0x08 | T12b |
| 0x16 | MIC_MUTE | state u8 | state read back | – | T17 |
| 0x17 | GET_VERSION | – | version 3 B, image sha256 32 B, board rev 1 B | – | T25 |
| 0x18 | FW_UPDATE_END | sha256 32 B | – | 0x08 | T25 |
| 0x40-0x49, 0x4F | provisioning block | section 7.3 | section 7.3 | 0x06, 0x09 | T6 |
| 0x4A | READ_CALIBRATION | offset u16 LE, length u8 | that slice of the stored calibration blob | 0x02 short payload, 0x06 nothing stored | T6 |
| 0x4B | ATECC_WRITE_CONFIG | block u8, mask 32 B, image 32 B (65 B with the opcode) | – | 0x02 short payload, 0x09 zone already locked | T6 |

*Added 2026-09-02.* The two rows above were missing. Both opcodes exist in `main.c` and both are sent by `provision.py`, so a table that stopped at `0x49` described neither the firmware nor the tool. `0x4A` READ_CALIBRATION is what makes TST-EEG-004 **T6's acceptance limit executable at all** – "constants read back byte-identical to those written" – because until it existed nothing in the package could read the calibration blob back. It is chunked because a full calibration set does not fit in one frame, and it is **exempt from the provisioning-mode gate**, because T6 runs after provisioning on a unit that has left that mode and may have had its zone locked. `0x4B` ATECC_WRITE_CONFIG writes one masked 32-byte block of the ATECC608B configuration zone; a word with an all-zero mask keeps its factory value.

**These two collided once and the collision is worth remembering.** On 2 September both `main.c` and `provision.py` allocated `0x4A` independently – the firmware to the calibration reader, the tool to the config write – because each took the next free opcode without the other. Had it shipped, step 7b would have sent a 3-byte read to a handler expecting 65 bytes and T6's read-back would have failed on every unit. `main.c` is the authority, `provision.py` moved its config write to `0x4B`, and `provision_selftest.py` now asserts that every `CMD_` constant in the tool is unique, which is the check whose absence allowed it. The interop harness carries the same guard from the other side: one of its 49 checks is "`0x4A` is still the calibration reader, not the config write".

SET_HP_LEVEL must clamp the codec volume register at the value recorded at calibration. RFQ E-29 caps the headphone output at 100 dB SPL at any commanded level, and the calculated full-scale output of the fitted codec and a 47 Ohm load is about 110 dB SPL, so the clamp is the only thing that meets the requirement. The clamp is **not implemented** today; the codec driver itself is a stub.

**Two corrections to this table, 2 September 2026.** 0x0F was FW_UPDATE_END here and CMD_IDENTIFY in main.c and in TOOL-EEG-022, and there was no 0x10 row at all against their CMD_LOOPBACK. A specification that collides with the source and with the tool written to it is worse than no specification, because each side can be checked against it and pass. The two connectivity commands keep the opcodes the source and the tool already use, and **FW_UPDATE_END moves from 0x0F to 0x18**, the first free opcode above the audio group. Nothing implements the firmware-update family yet, so the move costs nothing; moving IDENTIFY instead would have broken a shipped tool. 0x0D FW_UPDATE_BEGIN and 0x0E FW_UPDATE_DATA keep their opcodes, which is what main.c's CMD_FW_UPDATE_BEGIN 0x0D already says.

main.c's CMD_PROVISION 0x0E is **retired**; 0x0E is FW_UPDATE_DATA and provisioning mode is entered with 0x40. The enum in main.c still carries the retired name and it should be deleted (FW-D10).

The opcodes main.c handles today are **0x01-0x05, 0x08-0x0B, 0x0F and 0x10**, eleven in all, plus the **thirteen** provisioning opcodes 0x40-0x4B and 0x4F, which handle_provision() implements in full. *Corrected 2026-09-02: this sentence said "the eleven provisioning opcodes 0x40-0x49 and 0x4F", which predates 0x4A and 0x4B.* The rest are FW-D10.

CMD_IDENTIFY's capability flags are bit 0 CDC, 1 WebUSB, 2 microSD mounted, 3 codec initialised, 4 ATECC608B present, 5 provisioned. Bits 3, 4 and 5 were wrong until 2 September: nothing ever set bit 3, so a working codec always read as absent, and bit 4 was set only when the configuration zone was locked, which answers "has this unit been provisioned" when a production tester is asking "is the part fitted". They are now drv_codec_ready(), drv_atecc_present() and drv_atecc_config_locked() respectively – three different questions with three different answers. TOOL-EEG-022 section 2.3 defines the same six bits and agrees.

---

## 7. End-of-line provisioning

This is the step that turns an assembled board into a serialised instrument. It is TST-EEG-004 **T6** and it runs **after** the characterisation steps T7, T10, T12 and T17, because it writes their results into the unit. Only T5a precedes it.

### 7.1 The station

One PC (Windows 11 or Ubuntu 22.04), Python 3.11, pyserial, cryptography. **Offline.** An ESD bench, a USB hub, and a barcode scanner for the board serial. No signing key and no network credential is present on this station or in the building (F-19). The unit is powered from its own battery.

Two different cables reach the unit at this station and they must not be confused. The firmware image goes in through the **DevKitC-1's own UART USB-C port**, reached through the MP-01 opening, because that port carries the auto-reset circuit (section 2.3). The provisioning session that follows runs over the instrument's own USB link, through the ADuM4160 module's host USB-C, which is what a participant will later use.

### 7.2 The command

```
python3 tools/provision.py --port /dev/ttyACM0 --serial TIOV-B-0007 \
        --calibration cal_TIOV-B-0007.json \
        --out records/TIOV-B-0007.json
```

The serial passed to `--serial` is a `TIOV-B-nnnn` string. **The format is defined once, in PKG-EEG-015 section 5, and this document cites it rather than restating it**; what belongs here is only what the script does with it. `provision.py` rejects any argument that does not match the registered format, writes the string unchanged into the USB `iSerialNumber`, the ATECC608B data zone and the provisioning record, and uses it as the `<unit_serial>` directory name of section 5.9, so the label, the descriptor, the card and the record cannot disagree.

`--dry-run` prints the whole sequence without touching a device, so the operator can be trained and the station validated before a board exists. `STATION` and `OPERATOR` environment variables are recorded into the output.

### 7.3 The steps

*Rewritten 2026-09-02, from the script.* This section used to be headed “The eleven steps” and claimed: “this table is the script’s step list, read out of `firmware/tools/provision.py`, and where the two ever differ the script is what runs.” The second half of that sentence was right and the first half was not. The table was **not** the script’s step list. It had no configuration-zone write at all, and it put the lock at step 8, after GenKey. The script writes and locks the zone at **2b and 2c**, before GenKey.

**The order in the old table could not have worked, and following it would have scrapped parts.** A factory-fresh ATECC608B will not generate a key into a slot whose configuration has not been written and locked, so step 3 fails on every unit; and step 8 would then lock whatever the part arrived with – Microchip’s default configuration – which is irreversible, destroys the breakout and produces no device identity. The write and the lock therefore run first. Old step 8 becomes the read-back that proves the lock took, which is what TST-EEG-004 T6’s acceptance limit actually asks for; old step 9 read the same state with the same `0x48`, so the two are printed as one step, `8/9`, rather than sending an identical command twice with nothing in between. A new step **7b** reads the calibration constants back and compares them, which is the other half of that limit.

Each step prints its name, then `ok`, `FAILED: <reason>` or `skipped (dry run)`. A failure writes the partial record and exits with status 1. Nothing is retried automatically, because two of these steps are irreversible. The step numbers below are the script’s own, including its lettered ones; where this table and the script ever differ again, **the script is what runs**.

| # | Step | Opcode | Writes | Irreversible | Gate |
|---|---|---|---|---|---|
| 1 | enter provisioning mode | `0x40` | – | no | refused with `0x02` if the zone is already locked – see section 6.2 |
| 2 | read the ATECC608B serial number | `0x43` | – | no | |

| # | Step | Opcode | Writes | Irreversible | Gate |
|---|---|---|---|---|---|
| 2b | write the configuration zone from the template | `0x4B`, once per 32-byte block whose mask is non-zero | the ATECC608B config zone, masked | no | only with `--write-config` |
| – | *stop* | – | – | – | if `--lock` was given and 2b did not write, the run **ends here** rather than locking a zone the station never wrote |
| 2c | lock the ATECC608B configuration zone | `0x47` | the config zone | **yes** | only with `--lock`, and only after the serial is typed back or passed with `--confirm-serial` |
| 3 | generate the P-256 key pair in slot 0 | `0x41` | the private key, inside the element | **yes** | 20 s deadline |
| 4 | read back the public key | `0x42` | – | no | 20 s deadline |
| 5 | write the USB vendor and product identifiers | `0x44` | NVS: `usb_ids`, vid u16 + pid u16 | no | |
| 6 | write the hardware revision | `0x45` | NVS: `hw_rev` | no | |
| 6b | write the unit serial into the device | `0x49` | NVS: `unit_serial`, the `TI0V-B-nnnn` string | no | |
| 7 | write the calibration constants | `0x46` | NVS: `calib`, the blob from `--calibration` | no | skipped with no `--calibration` |

| # | Step | Opcode | Writes | Irreversible | Gate |
|---|---|---|---|---|---|
| 7b | read the calibration constants back and compare | `0x4A`, chunked | – | no | **this is TST-EEG-004 T6's acceptance limit**: byte-identical or the step fails |
| 8/9 | read the state back and confirm the lock | `0x48` | – | no | if `--lock` was given and the zone reads unlocked, the operator is told to hold the unit and report it |
| 10 | leave provisioning mode | `0x4F` | reboot; `iSerialNumber` is now the `TIOV-B-nnnn` written at 6b | no | |

**Both irreversible steps are behind flags.** Neither `--write-config` nor `--lock` is implied. Without `--lock` nothing in a run is irreversible and the run cannot produce a finished unit, which is what a training or station-validation run wants. `--serial` is checked against the PKG-EEG-015 section 5 format before anything is sent.

The private key is generated **inside** the secure element and cannot be read out by anyone, including the programme. Step 2c cannot be undone: a config zone locked with the wrong template scraps the ATECC breakout. J11 is socketed, so the loss is one breakout and not one board, and the manufacturer is required to hold 10 % spare ATECC608B breakouts.

**Where this document and the script disagree about NVS, the firmware is the fact.** The script's own header still says the identity and the constants land in namespace `"tiov"` in the **default** `nvs` partition, and warns that `drivers.c` erases that partition unconditionally on `ESP_ERR_NVS_NO_FREE_PAGES`. That was true and **is not true any more**: `drivers.c` now routes the `calib` key to partition `calib` / namespace `eegcal` and every identity key to `prov` / `eegcfg`, initialising each on first use, which is what section 2.5 specifies. The stale warning is in `provision.py` and in `README_provisioning.md`, neither of which this document owns; it is raised in section 10.

**The whole of section 7 is still unexecuted.** No step above has ever been sent to a real ATECC608B. The interop harness drives `0x4A`, `0x4B` and the mode gate against a simulated part, which proves the framing and the refusals and nothing about the silicon.

**The ATECC608B-TNGTLS variant must never be substituted.** It arrives with its config zone already locked and its keys owned by Microchip; F-18 and E-21 are impossible on it. The AVL alternate column must say so.

No eFuse is burned during provisioning on Phase 1. Section 7.5 says when eFuses are burned and when they are not.

### 7.4 What the script prints and what it returns

On success it prints the unit serial, the ATECC serial and the public-key fingerprint, then the instruction to print the fingerprint on the enclosure label (M-03) and return the record with the unit. The record is `records/<serial>.json`:

```
{
  "unit_serial": "TIOV-B-0007",
  "hardware_revision": "B",
  "board": "EEG-CAR-01 Rev B",
  "provisioned_utc": "2026-09-01T09:14:02Z",
  "station": "...", "operator": "...",
  "usb_vid": "0x1209", "usb_pid": "0x0001",
  "atecc_serial": "0123...",
  "device_public_key": "<128 hex chars, raw uncompressed X||Y>",
  "public_key_fingerprint": "A1B2 C3D4 E5F6 0718",
  "provision_state": "...",
  "calibration": { ... },
  "steps": [ {"step": "...", "result": "pass"}, ... ]
}
```

**The public-key fingerprint is defined here and nowhere else.** It is the **first 8 bytes of SHA-256 over the 64-byte uncompressed public key**, rendered as **16 uppercase hexadecimal characters in four groups of four separated by single spaces**, for example `A1B2 C3D4 E5F6 0718`. ASM-EEG-007, QP-EEG-010, PKG-EEG-015, TST-EEG-004 and REG-EEG-012 cite this definition and do not restate it. The fingerprint is what goes on the M-03 label, and the label is generated from this record so the two cannot disagree. All records for a shipment are returned as one manifest.

### 7.5 How the signing key never leaves the programme, and when eFuses are burned

| Who | Holds | Does |
|---|---|---|
| Programme, Brussels | the RSA-3072 secure-boot private key, offline, two custodians, two backups | builds, signs with `espsecure.py sign_data`, publishes the release |
| Manufacturer | pre-signed binaries and the public-key **digest** file only | flashes, provisions, tests, and from Phase 2 burns eFuses |

**Phase decision, written down and not deferred: no eFuses are burned on the two Phase 1 prototypes.** They ship with secure boot and flash encryption disabled and run unsigned images, so the firmware volunteer can re-flash them freely during bring-up. This is a scoped deviation from F-19 for two units that are never given to a participant. **TST-EEG-004 T25 is therefore a Phase 2-onward step and is not a Phase 1 gate**; it is not run on the prototypes and its absence is not a Phase 1 non-conformance.

From Phase 2 onward the burn happens at the station, in this order, each step irreversible:

1. `espefuse.py burn_key_digest secure_boot_digest.bin BLOCK_KEY0 SECURE_BOOT_DIGEST0`
2. `espefuse.py burn_efuse SECURE_BOOT_EN 1`
3. first boot: the bootloader generates the flash-encryption key **on the device**, so the manufacturer never holds that either
4. `espefuse.py burn_efuse DIS_DOWNLOAD_MANUAL_ENCRYPT SOFT_DIS_JTAG`

5. `espefuse.py summary` captured verbatim into the per-unit record

TST-EEG-004 places that burn after T5 to T17 and before T18, and requires one deliberate negative test per batch: attempt to boot an unsigned image and confirm rejection (T25).

Once burned, the ESP32-S3 module **is** part of the instrument identity, together with the ATECC608B at J11. It is not a field-swappable part, notwithstanding the general module philosophy of DSN-EEG-003 section 2. A failed MCU module means the unit returns to the programme for re-provisioning under a new record. On the Phase 1 prototypes, where nothing is burned, the module remains swappable.

---

### 7.6 The ATECC608B configuration zone, byte by byte

**Status: reviewed here, NOT approved.** This section is the review a person signs off; it does not sign itself off. `firmware/tools/atecc608b_config.py` carries `REVIEWED = False`, and `provision.py` **refuses to run against a real part** while it is False — see 7.6.5.

#### *7.6.1 The framing, which is not "4 of 128 bytes are done"*

It is commonly stated that four of the 128 configuration bytes have been reviewed and 124 have not. That is the wrong shape and it makes the remaining work look like a filling-in exercise.

The template **specifies four bytes and deliberately does not write the other 124.** It emits an image *and a mask*; the mask is `0xFF` on four offsets and `0x00` everywhere else, and a writer consults the mask, not the image. The 124 unspecified bytes are not blanks awaiting a value — they are bytes this programme has decided **not to have an opinion about**, so that the part keeps whatever it shipped with.

So there are two different questions, and only the first is "what value?":

1. **The four specified bytes** — are these the right values? (7.6.2)
2. **The 124 unspecified bytes** — is *inheriting the factory default* an acceptable posture on a part that will be permanently locked? (7.6.3)

Question 2 is not answered by choosing 124 values. Choosing them would be the *worse* outcome: every one would be a number invented here and locked into silicon, and the reviewer would have 128 things to check instead of four.

#### *7.6.2 The four specified bytes*

`SLOT_KEY = 0`. The instrument uses exactly one slot.

| Offset | Field | Value | Verified? |
|---|---|---|---|
| 20 | SlotConfig[0] low | `0x81` | reasoned; bit meanings **asserted**, see below |
| 21 | SlotConfig[0] high | `0x20` | **UNVERIFIED — checklist item 1** |
| 96 | KeyConfig[0] low | `0x33` | reasoned |

| Offset | Field | Value | Verified? |
|---|---|---|---|
| 97 | KeyConfig[0] high | `0x00` | reasoned |

**Offset 20 — SlotConfig[0] low.** `0x81` = `IsSecret` (bit 7) set, `EncryptRead` (bit 6) clear, `LimitedUse` (bit 5) clear, `NoMac` (bit 4) clear, `ReadKey` (bits 3–0) = `0x1`.

- `IsSecret = 1` with `EncryptRead = 0` is the whole reason the part is fitted: the private key is not readable by any means. This is E-21 and it is the least ambiguous bit here.
- `LimitedUse = 0` is **load-bearing and correctly clear.** A limited-use slot burns a monotonic counter per use. F-08 signs every 2048 samples — about one signature every two seconds at 1000 Hz — so any counter would exhaust and the unit would stop signing mid-study. Setting this bit would be a slow, silent field failure.
- `ReadKey = 0x1`. For an ECC private-key slot these four bits are permissions rather than a key id: bit 0 external signatures, bit 1 internal signatures, bit 2 ECDH, bit 3 ECDH master secret out in the clear. Only bit 0 is set, which is the minimum F-08 and T16 need, and bits 2 and 3 are the ones that would matter if wrong — this design does no key agreement at all, so granting ECDH would only create a way to use the identity key for something the instrument does not do. **The bit-position assignment above is asserted from the published semantics of the family, not read out of the 608B datasheet in front of a reviewer. It is checklist item 2 and it stays open.**

**Offset 21 — SlotConfig[0] high. This is the one that can scrap parts, and it is not resolved here.** `0x20` = `WriteKey` (bits 11–8) = `0x0`, `WriteConfig` (bits 15–12) = `0x2`.

The *intent* is exact and is not in doubt: **GenKey may create the key inside the part; no command may write a private key in from outside.** What is in doubt is whether `0x2` encodes that intent on the 608B.

What a reviewer has to establish, and why each half matters:

- **Too permissive** — if the encoding leaves an external write path open, a key can be loaded from outside and E-21 is defeated. The failure is silent: the part works, the signatures verify, and the guarantee that the key never existed outside the device is gone.
- **Too restrictive** — if the encoding forbids GenKey, then after the configuration zone is locked GenKey is refused for ever, provisioning step 3 fails, and the part is scrap. The failure is loud but terminal, and it happens **after** the irreversible step.

The two bits usually described as `GenKeyEn` and `PrivWrite` within this nibble are the ones that decide it, and the upper two bits additionally select the plain `Write` command's behaviour. `IsSecret = 1` on an ECC private slot already bars the ordinary `Write` path, so the nibble's `Write` half is expected to be moot here — **expected, not established.**

> **This must be read off the SlotConfig `WriteConfig` bit table in the ATECC608B datasheet specifically, not the ATECC508A's, and not from recollection — including the recollection in this paragraph.** Nothing in this package has opened that table. The part number is ATECC608B-SSHDA (checklist item 7).

**Offsets 96 and 97 — KeyConfig[0].** `0x0033` = `Private` (bit 0) set, `PubInfo` (bit 1) set, `KeyType` (bits 4–2) = `0x4` (P-256), `Lockable` (bit 5) set, `ReqRandom`, `ReqAuth`, `AuthKey`, `PersistentDisable`, `X509id` all clear.

- `Private = 1`, `KeyType = P-256` is the F-18 device key, and matches `drivers.c` calling GenKey mode `0x04` to create and mode `0x00` to read the public half.
- **`PubInfo = 1` is load-bearing.** With it clear, GenKey mode `0x00` — provisioning step 4, "read back the public key" — is refused, and the unit ends up with a key nobody can name. The M-03 label fingerprint, the Data Matrix and T16 all derive from that read-back. This is the bit whose absence would be discovered only after the zone was locked.
- `ReqAuth = 0`, `AuthKey = 0` is correct **because of a fact about this programme, not a preference**: F-19 puts no host secret anywhere in the building, so there is no second key to authorise against. An authorisation requirement here would make the key unusable rather than safer.
- `Lockable = 1` costs nothing and leaves open the option of sealing the slot individually once the key exists. No opcode in this package uses it.

### *7.6.3 The 124 bytes that are not written*

Grouped by what inheriting the factory default actually commits the programme to.

| Offset | Region | Posture | Assessment |
|---|---|---|---|
| 0–15 | Serial number, revision, AES/I²C enable | **read-only** | Not a decision. The part refuses writes here. Safe by construction |
| 16 | I²C address | inherited | **Safe, and checked**: factory default `0xC0` is 7-bit `0x60`, which is the address `drivers.c` already uses. Verified against the firmware, not merely assumed |
| 17 | Reserved | inherited | Not a decision |
| 18 | CountMatch | inherited | No slot here is LimitedUse and no counter is consumed, so the count-match feature is inert. **Low risk, unexamined** |
| 19 | ChipMode (watchdog, TTL enable) | inherited | **OPEN — checklist item 6.** The longest command the firmware issues is GenKey, for which `drivers.c` allows 115 ms. A watchdog that expires mid-GenKey on a locked part is not recoverable by retrying. The default must be confirmed to exceed worst-case GenKey time |

| Offset | Region | Posture | Assessment |
|---|---|---|---|
| 20–51 | SlotConfig[1..15] | inherited | **OPEN — checklist item 5.** Fifteen slots this instrument never uses, left at whatever posture the factory chose, and then locked permanently. The question is not “what should they be” but “is the default an acceptable posture on a locked part, or must unused slots be made explicitly unusable” |
| 52–67 | Counter[0..1] | inherited | Not a decision. No LimitedUse slot, so no counter is consumed |
| 84–85 | UserExtra, UserExtraAdd | inherited | Writable after lock by design; not part of the key posture |
| 86–87 | LockValue, LockConfig | **never written by a template** | Correct and important: these are set by the `Lock` command (`0x17`), which is provisioning step 2c. A template that wrote them would be attempting the lock as a side effect of a configuration write |
| 88–89 | SlotLocked | inherited | Individual slot locking is not used by any opcode in this package |
| 96–127 | KeyConfig[1..15] | inherited | Same question as SlotConfig[1..15] — checklist item 5 |

**A gap in the template’s own byte map, found in this review.** The map in `atecc608b_config.py` names offsets 0–15, 16, 19, 20–51, 52–67, 84–87, 88–89 and 96–127. On the **608**, the region between the counters and UserExtra, and the pair just before KeyConfig, carry device features the 508A did not have — the published 608 map places `UseLock`, `VolatileKeyPermission`, `SecureBoot`, the KDF IV controls and `ChipOptions` in offsets **68–74 and 90–91**. **The template’s map does not name any of them.**

That matters because `SecureBoot` and `ChipOptions` are not inert: they configure device behaviour that is then locked permanently along with everything else. This does not necessarily mean the defaults are wrong — it means **nobody has looked**, and a byte nobody has looked at is exactly what this review exists to surface.

> **This offset attribution is asserted from the published 608 configuration map and has not been read out of the datasheet.** Confirm the offsets, the factory defaults and whether inheriting them is acceptable. **This is a new checklist item — item 9 — and it is open.**

### *7.6.4 What is unresolved, named as unresolved*

Nothing in 7.6.2 or 7.6.3 closes any checklist item. The full open list is section 6 of `firmware/tools/ATECC608B_CONFIG_TEMPLATE.md`, items 1–8, plus:

| Item | What is unresolved | Consequence if wrong |
|---|---|---|
| 1 | `WriteConfig` nibble encoding (offset 21) | E-21 defeated, or the part scrapped after the lock |
| 2 | `ReadKey` bit meanings for an ECC private slot | an unintended permission locked in |
| 3 | The `0x4A` opcode framing | host and firmware must then change together |
| 4 | Whether `Sign` needs the **data zone** locked as well | **if it does and it is missing, T16 cannot pass on any unit** — a firmware gap, not a template gap. `drivers.c` exposes only `drv_atecc_lock_config()` |
| 5 | Posture of the fifteen unused slots | fifteen slots locked at an unexamined default |
| 6 | ChipMode watchdog vs worst-case GenKey | unrecoverable timeout on a locked part |
| 7 | Part number is **-SSHDA**, never **-TNGTLS** | TNGTLS arrives pre-locked with Microchip's keys: F-18 and E-21 are impossible on it |
| 8 | One part written, read back, locked, keyed, verified end to end | until this has happened, none of the above is more than reasoning |
| **9** | **608-specific bytes 68–74 and 90–91, unnamed in the template's map** | device features locked at an unexamined default |

**Item 4 is the one that is not a template question at all.** If `Sign` requires the data zone locked, provisioning needs a further irreversible step and a further opcode that does not exist anywhere in the firmware. It should be settled before the others, because it changes what provisioning *is*, not merely what it writes.

### *7.6.5 The guard, and why it cannot be turned on by accident*

`firmware/tools/atecc608b_config.py` carries a review block: `REVIEWED`, `REVIEWED_BY`, `REVIEWED_AGAINST` and `CHECKLIST_CLOSED`. `review_status()` reports `ok` **only** when a person has set the flag, named themselves, named the document they read, and closed every checklist item. A partially filled block is not a review and is reported as one that is not.

`provision.py` calls it before anything else and, on any run that is not `--dry-run`, **refuses and exits 2**:

```
REFUSING TO RUN AGAINST A REAL PART.

  The ATECC608B configuration zone is not marked reviewed:
    - REVIEWED is False in atecc608b_config.py
    - REVIEWED_BY names nobody
    - REVIEWED_AGAINST names no datasheet
    - checklist items still open: 1, 2, 3, 4, 5, 6, 7, 8
```

Three properties of the guard are deliberate:

1. **There is no `--force`, no environment variable and no file that enables it.** The only mechanism is a person editing the source. A reviewed flag a script can set is a reviewed flag a tired operator can set at 2 a.m. with a tray of parts in front of them.
2. **It is not inferred from anything** — not from the presence of the `.bin`, not from its checksum, not from a record of a previous run.
3. **`--dry-run` is unaffected**, because it opens no port and writes to no silicon. The station, the record format and `provision_selftest.py` can all be exercised while the review is outstanding. `provision_selftest.py` passes unchanged.

**Consequence to state plainly to assemblers:** provisioning cannot be quoted as a production operation today. It is blocked at the tool, by design, until a named person closes the checklist.

---

## 8. The host verification tool

`firmware/tools/verify_stream.py` is the decoder the production test uses, and the same tool the programme uses to read a returned microSD card, laid out as section 5.9 defines. It reads from a file or a serial port, and reports on the four failures that are otherwise invisible.

```
python3 verify_stream.py --file session.bin --pubkey records/TIOV-B-0007.json
python3 verify_stream.py --port /dev/ttyACM0 --seconds 60 --pubkey ... --json t14.json
```

| Check | How |
|---|---|
| corrupted frame | CRC-32 over the decoded body, counted as `BAD_CRC` |
| lost frame | sequence continuity modulo 65536, and GAP frames |
| discontinuous timeline | `first_sample` continuity against `n_samples`, resuming after a GAP |
| unverifiable block | ECDSA P-256 over the chained digest, against the exported public key |

It prints one JSON report and exits **0** on pass, **2** on fail, so it can be a test step directly:

```
{
  "frames": {"DATA": 90000, "STATUS": 1800, "SIGNATURE": 43},
  "samples": 1800000, "sample_rate_hz": 1000, "duration_s": 1800.0,
```

```
  "first_sample_index": 41, "last_sample_index": 1800041,
  "gap_frames": 0, "samples_lost_to_gaps": 0,
  "signature_blocks": 43, "signature_blocks_verified": 43,
  "errors": [], "error_count": 0, "pass": true
}
```

| Step | Use | Pass condition |
|---|---|---|
| **T5** | `--port --seconds 10` on each of Windows 11, macOS and Linux, after the OS has enumerated the device | frames decode with zero bad CRC on every host. The descriptor dump itself comes from `pnputil /enum-devices`, `ioreg -p IOUSB` and `lsusb -v`, not from this tool |
| **T13** | `--port` capturing while `CMD_TIMING_SELFTEST` runs | zero CRC or continuity errors across the 40 tones; the median and p95 come from the CMD_ACK |
| **T14** | `--file` on the 30-minute host capture and `--file` on the card copy, then compare the two reports | `DATA` = 90,000, `error_count` = 0, and the two reports identical |
| **T16** | `--file --pubkey records/<serial>.json` over a session of at least 8192 samples | `signature_blocks_verified` = `signature_blocks`, and **`signature_blocks` > 0** |

That last condition has to be asserted separately by the operator: the tool's `pass` flag is true when `blocks == 0`, so a stream carrying no SIGNATURE frames at all would pass. This is stated rather than hidden, and the fix is listed in section 10.

The T-numbers above are TST-EEG-004 Rev C's and are cited, not coined. Where a step this document needs does not exist there, it is raised as an open item rather than numbered here.

### 8.1 The protocol interoperability harness

`webtest/tests/interop/` is a second host-side tool, added on 2 September 2026, and it answers one question: **do the firmware and the browser test program speak the same protocol?** Run it with `sh webtest/tests/interop/run.sh`. It needs a C compiler and Node 18 or later, and it needs neither ESP-IDF nor hardware.

It compiles the shipped `firmware/main/main.c` against small ESP-IDF stubs, simulates the peripherals in `drivers_sim.c`, wires the two USB endpoints to stdin and stdout, and drives the result with `webtest/js/protocol.js` – the same module the browser tool ships. It reports **57 checks, all passing** as this document is issued. *Corrected 2026-09-02: this paragraph said 32, which was the count before the provisioning-acknowledgement and opcode work of that day.* The checks are frame decode and CRC, IDENTIFY parsed by the host's own decoder including all six capability bits and the 6 MiB ring of FW-D13, LOOPBACK at four payload lengths, opcode echo on seven commands and the twelve-byte CLOCK_XCHG result, both halves of the S-01 interlock, silence in reply to a corrupted CRC and to random noise, and – new on 2 September – the provisioning family: a config write refused outside

provisioning mode, opcode echo on `0x4B`, a well-formed config write accepted after `0x40`, a short config write refused with bad length, and **“`0x4A` is still the calibration reader, not the config write”**, which is the guard against the opcode collision described in section 6.3 recurring.

It exists because the test that came before it could not have found FW-D14. That test drove the real host code against a device **simulated in JavaScript**, and the simulation was written by hand from this specification, as `main.c` was. The two drifted from it independently and in different directions – `main.c` dispatched on the frame’s first byte, the simulation answered `{opcode, 0xFF}`, and section 6.2 specified a third shape – and every test passed throughout, because the JavaScript host and the JavaScript device shared one misunderstanding. A simulator written from the same document as the firmware tests the pair of readings, not the document.

What it does **not** do: it says nothing about the analogue front end, the converters, the timing, the descriptors (the stubs expand the TinyUSB macros to placeholders, and `run.sh` neutralises the `_Static_assert` of FW-D11 in its private copy), or anything else that needs a board. It retires no row of section 1.3 that needs a unit.

*Corrected 2026-09-02.* This paragraph used to end “and FW-D20 is a defect it looks straight at and does not see.” That was true of the 32-check version, which checked only that `0x48` echoed its own opcode – a test the wrong ack shape passes. The provisioning checks added on 2 September read a **status** at the section 6.2 offset, and one of them expects a specific `0x02` bad-length refusal, so the shape is now covered from the host side. It is worth keeping the old sentence visible: a harness can look straight at a defect and pass, and the fix was to make it read the field the defect was in.

### 8.2 The QEMU run, and exactly what it proves

*New, 2 September 2026.* The firmware has been **run**, once, under `qemu-system-xtensa 9.0.0 (esp-develop)` with `-M esp32s3 -m 4M`, against a `sdkconfig.defaults` + `sdkconfig.phase1` + `sdkconfig.qemu` build. The complete console capture is `firmware/release/qemu_boot.log` and it is part of the release.

**This is an emulator, not a unit, and the distinction is the whole of what follows.** QEMU’s `esp32s3` machine has no octal PSRAM, no microSD, no ES8388 and no ADS1299. Every peripheral this instrument exists to talk to is absent.

What the run **does** prove – the part that needs no peripheral:

| Observed in the log | What it settles |
|---|---|
| ROM loader, then `ESP-IDF v5.2.5 2nd stage bootloader` | the built bootloader is a bootloader and this is the version that built it |
| the nine-row partition dump, matching `partitions.csv` offset for offset | the flashed table is **this** table, and `nvs`, `otadata`, `phy_init`, `factory`, `ota_0`, `ota_1`, `calib`, `prov` and `storage` all land where section 2.5 says |
| `Defaulting to factory image`, five segments loaded, `Loaded app from partition at offset 0x20000` | the image is loadable from the `factory` slot at the section 9 offset, and its segment map is consistent |

| Observed in the log | What it settles |
|---|---|
| `Project name: eeg_field_kit`, `App version: e91f9d58-dirty` | the application descriptor carries the CMake project name of section 9 – and a **dirty** tree, which is why section 2.1 says the build is not yet reproducible |
| `main_task: Calling app_main()` then the firmware's own first log line | `app_main()` is reached and runs |
| `drv_sd_init()` and `drv_codec_init()` both time out, log, and **continue** | the two degrade-gracefully paths behave as written: a unit with no card or no codec does not refuse to start |
| the FW-D13 diagnostic, verbatim, then `abort()` | the ring-buffer guard added on 2 September fires and names the cause, rather than asserting somewhere inside `xRingbufferSend` a frame later |

What it proves **nothing** about: every register value, the daisy-chain order, the SPI timing, the ADS1299 entirely, the USB descriptors and enumeration, the contact lights, the ATECC608B, the fuel gauge, timing accuracy, and anything else in TST-EEG-004. No row of section 1.3 is retired by it and no requirement is met by it.

**The abort at the end is the correct behaviour, not a failure of the run.** PSRAM is absent, so the 6 MiB ring of F-06 cannot be allocated, and `app_main()` refuses to carry on without it. `sdkconfig.qemu` sets `CONFIG_SPIRAM_IGNORE_NOTFOUND=y` only so the boot gets far enough to reach that point instead of failing earlier in `cpu_start`; it is not a configuration for any unit (section 2.4).

**One thing the log raises that is not a QEMU artefact.** A unit that boot-loops or aborts in a participant's home cannot be told apart from a dead battery or a bad cable, and the browser tool cannot ask it what is wrong because it never enumerates. The abort is right for a bench and wrong for a field device, and what a field device should do instead – enumerate, report the fault in STATUS, and refuse to record – is not written. That is carried in section 10.

---

## 9. Firmware release process

**Version scheme.** `FW-EEG-001 v<major>.<minor>.<patch>`, carried in the CMake `project(... VERSION ...)` field, in the ESP-IDF application descriptor, in the STATUS frame and in `CMD_GET_VERSION`. The shipped project declares **0.2.0**. `v0.x` is Phase 1 bring-up; `v1.0.0` is the first fleet release and is gated on the safety review sign-off, which has not happened.

**Reproducible build.** `FROM espressif/idf:v5.2.5` pinned by image digest, `SOURCE_DATE_EPOCH` set from the release commit date, one `make release`. The manufacturer is invited, not required, to reproduce the hash. **This has not yet been done.** The images described below were built on a developer machine from a working tree with uncommitted changes – the boot log records the application version as `e91f9d58-dirty` – so the SHA-256 list below is the hash of *those* images and not yet a hash anyone else can reproduce. Rebuilding from a clean tree in the pinned container is an open item.

**Release contents, flash offsets and what was actually built.** *Measured 2026-09-02.* The four files below are in `firmware/release/` with a `manifest.json` carrying their SHA-256; the figures are `idf.py size` and the files on disk.

| File | Offset | Bytes | SHA-256 (first 16 hex) |
|---|---|---|---|
| `bootloader.bin` | `0x0` | 23,168 | `23864fcd3a087b98` |
| `partition-table.bin` | `0x8000` | 3,072 | `fdd6a7170583bbaf` |
| `ota_data_initial.bin` | `0xF000` | 8,192 | `7d2c7ac4888bfd75` |
| `eeg_field_kit.bin` | `0x20000` (the `factory` slot) | 405,360 | `90e5ec1f6b91fb4d` |

`manifest.json` carries the full 64-character digests and is the file the manufacturer verifies against; the truncations above are for reading, not for checking.

**Memory, and the one figure that is a problem.** *Added 2026-09-02; the whole of this paragraph is new, and item 17 of section 10 is the open half of it.*

| Figure | Value | Reading |
|---|---|---|
| Total image, as `idf.py size` reports it | **405,245 bytes** | 13 % of the 3 MB `factory` slot. Ample |
| `eeg_field_kit.bin` on disk | **405,360 bytes** | the same image plus the padding and appended SHA-256 that `esptool` writes into the flashable file |
| Flash code / rodata | 237,099 / 75,468 bytes | |
| DIRAM used | 88,799 of 345,856 bytes (25.7 %) | 257,057 bytes remain. Ample |
| **Static IRAM used** | **16,383 of 16,384 bytes – one byte free** | **not a pass.** See below |

The IRAM figure is the finding. One byte free is a cliff, not a margin: the next function anyone marks `IRAM_ATTR` fails the link with an error naming a section rather than a cause, and this design has more interrupt work coming – E-13's tone scheduler and E-12's onset detector are both interrupt-side work that does not exist yet.

Two ISRs that this firmware does not use were taken out of IRAM to make room – `CONFIG_SPI_SLAVE_ISR_IN_IRAM=n` and `CONFIG_GPTIMER_ISR_HANDLER_IN_IRAM=n`; there is no SPI slave anywhere in the design and nothing uses `gptimer`. **It did not help.** The figure came back 16,383 of 16,384, byte for byte. The change is kept, because carrying interrupt handlers for a bus and a timer this firmware does not use is wrong either way, but **it is not the fix and must not be recorded as one.** SPI *master* stays in IRAM deliberately: `sample_task()` reads the converters on every DRDY and that path must survive a flash-cache stall.

What is not yet known is whether the pool is genuinely full or whether `esp_idf_size` is reporting against a fixed 16 kB window that is not the real limit on an ESP32-S3 with octal SPIRAM and XIP. Settling that needs the linker map read against hardware. It is carried as section 10 item 17 and as SIM-EEG-018 open item 1, and it is **not** guessed at from here.

*`storage.fat` was a fifth row here and is withdrawn: no such file exists in the package and nothing in the firmware mounts the internal `storage` partition it would have been written to. See the note in section 6.1.*

```
esptool.py --chip esp32s3 --port <devkit UART port> --baud 921600
write_flash \
  --flash_mode qio --flash_size 16MB \
  0x0 bootloader.bin 0x8000 partition-table.bin 0xF000
ota_data_initial.bin \
  0x20000 eeg_field_kit.bin
```

`<devkit UART port>` is the DevKitC-1's own UART USB-C port, per section 2.3. `esptool.py` drives the DevKit's auto-reset circuit itself; there is no separate boot-mode step and no fixture on J26.

`manifest.json` accompanies every release: version, ESP-IDF commit, `esp_tinyusb` version, build container digest, SHA-256 of every file, the WinUSB GUID, the `board_pins.h` regeneration record, and the secure-boot public key digest. The manufacturer verifies the SHA-256 list before flashing and records **which version and which image hash went onto which unit** into that unit's record. A circulating kit that visits twenty participants will be re-flashed several times over its life; without that record nobody can say what a given recording was produced by, and a pre-registered study cannot tolerate that.

**A/B update with rollback (F-20).** `ota_0` and `ota_1` alternate; `factory` is the recovery image, flashed once at end of line and never overwritten in the field. A new image arrives over `CMD_FW_UPDATE_BEGIN` / `DATA` / `END`, is written to the inactive slot, and boots in `PENDING_VERIFY`. `esp_ota_mark_app_valid_cancel_rollback()` is called **only** after three self-checks pass: the device enumerated on USB, the ATECC608B serial was read, and one ADS1299 RDATAC read returned. If any check fails or the unit resets first, the bootloader reverts to the previous slot on its own. Anti-rollback is on from Phase 2, so a downgrade below the burned secure version is refused; on the Phase 1 prototypes, which burn no eFuses, there is no anti-rollback and a downgrade is possible.

The manufacturer never builds firmware and never modifies source. Any change goes through the programme and produces a new version.

---

## 10. Open items, honestly

| # | Item | Consequence | Owner | Gate |
|---|---|---|---|---|
| 1 | ~~**The firmware has never been compiled against a real ESP-IDF installation.**~~ **CLOSED 2026-09-02.** It was built against ESP-IDF v5.2.5, clean at ESP-IDF's default `-Wall -Werror=all`; images in `firmware/release/`. Five project defects surfaced in the process and are recorded in sections 2.2 and 2.4 | Every ESP-IDF header, Kconfig option and TinyUSB macro this source names now resolves, and both `_Static_asserts` held. It says nothing about whether any of it is *correct* | firmware volunteer | closed. What replaces it is items 1a and 2 |
| 1a | **The build is not reproducible yet.** It was made on a developer machine from a dirty tree, not in the pinned `espressif/idf:v5.2.5` container from a release commit | The SHA-256 list in section 9 cannot be independently reproduced, and F-19's "the manufacturer verifies before flashing" is a weaker guarantee than it reads | programme | a clean-tree container build whose hashes match |
| 2 | **It has never run on hardware.** It has run **once under QEMU** (section 8.2), which is a much weaker claim: QEMU's `esp32s3` has no PSRAM, no microSD, no ES8388 and no ADS1299, so no peripheral this instrument uses was present | Every register value, the daisy order, the SPI timing and the descriptor set are assumptions. The QEMU run retires no row of section 1.3 | firmware volunteer | T5, then T13 |

| # | Item | Consequence | Owner | Gate |
|---|---|---|---|---|
| 3 | **The five drivers are written and none has run.** `drivers.c` implements them (section 1.2); no I2C address, register write or timeout in it has ever been answered by a part. The block-signing task is still not written and not declared | TST-EEG-004 steps **T12, T13, T14, T16 and T17** cannot be signed off on code inspection, and T16 has no input at all while nothing emits SIGNATURE frames. The total step count is TST-EEG-004 Rev C's to state, not this document's | firmware volunteer | each driver has a named step |
| 4 | ~~**The contact-light phase driver does not exist**~~ **CLOSED 2026-09-02** (section 3.2, FW-D16). `lights_phase()` and `lights_task()` drive LED_V against the shift register in antiphase, and the three colours come from both halves of the lead-off measurement. **E-27 is met in the source.** The alternation is about 250 Hz, not the nominal 240, because it quantises to the 1 kHz tick; the requirement is "above 100 Hz" | Written, never run. T11 is no longer blocked by a missing driver; it still needs a unit and a colorimeter. What remains open is narrower: `CMD_LIGHTS` modes 2, 3 and 4 – forcing a colour – are not implemented and the source answers `0x00` OK rather than `0x0B` to them | firmware volunteer | T11 |

| # | Item | Consequence | Owner | Gate |
|---|---|---|---|---|
| 5 | ~~Descriptor macro names … are unverified against `esp_tinyusb ~1.4.2`~~ **CLOSED 2026-09-02.** They resolved against `esp_tinyusb` 1.4.5, and the `_Static_assert` on the MS OS 2.0 descriptor length (`main.c:705`) was evaluated and held | The descriptor set assembles at the asserted length. Whether Windows binds WinUSB to it is a different question and is T5 | firmware volunteer | T5 on Windows 11 |
| 6 | USB **PID is a placeholder** and the WinUSB GUID is the TinyUSB example GUID | Blocks the fleet, not the two prototypes. Browser authorisation is keyed to VID, PID and serial | programme | pid.codes allocation |
| 7 | ~~`sdkconfig.defaults` is **still missing five lines**~~ **CLOSED 2026-09-02** (section 2.4). All five are in the file, and one of them was named wrongly here: the key is `CONFIG_TINYUSB_VENDOR_COUNT`, not `CONFIG_TINYUSB_VENDOR_ENABLED`, which does not exist and which ESP-IDF accepted in silence | The vendor class is enabled and the WebUSB interface exists in the built image. **One half is still open**: `CONFIG_ESP_TASK_WDT_EN=y` is set but nothing subscribes `sample_task` or the SD path to the watchdog, so a stalled SD write still does not reset the unit | firmware volunteer | T5; the watchdog subscription before T14 |

| # | Item | Consequence | Owner | Gate |
|---|---|---|---|---|
| 8 | **F-06 has been relaxed by ECO-EEG-025** to 90 s of ring plus microSD backfill; the 6 MiB ring gives 126 s as the simulator counts it, 124 s over the framed stream, and meets it either way (section 5.8) | Carried by RFQ-EEG-001 Rev E, DSN-EEG-003 Rev D and TST-EEG-004 Rev C. `RING_BYTES` is now 6 MiB in `main.c` and FW-D13 is closed. The residual work is the **retransmit index of FW-D06**, which does not exist: `CMD_RETRANSMIT` still ignores its range and drains the ring destructively, so the ring is allocated but the backfill the 90 s depends on cannot yet be delivered | firmware volunteer | T15 |
| 9 | `verify_stream.py` has **no `--session-id`** and roots the signature chain at 32 zero bytes; TST-EEG-004 T16 says the chain is rooted at the session identifier | Until the option is added, T16 must be run with an all-zero root and the deviation recorded | programme | before T16 is first executed |
| 10 | `verify_stream.py` reports `pass` when there are **no SIGNATURE frames at all**, and has no `--compare` for T14's card-versus-host check | The operator must assert `signature_blocks > 0` and diff the two reports by hand | programme | before T14 and T16 |

| # | Item | Consequence | Owner | Gate |
|---|---|---|---|---|
| 11 | The **microSD file format** is now defined in full in **section 5.9** and this document owns it; SVC-EEG-013 section 2 R2 cites that section and its `/SESSIONS/...eegs` layout with a 512-byte header is withdrawn. What is still open is the code. *Corrected 2026-09-02: this row said `sd_append()` and `sd_free_mb()` "are stubs"; they are written (`drivers.c:163` and `:175`) and have never addressed a card.* Nothing writes the sidecar, and `verify_stream.py` neither reads a sidecar nor checks the file hash | The definition no longer differs between documents, but no card has ever been written, so the layout is unexercised and T14 cannot run | firmware volunteer | T14 |
| 12 | The **host tool names diverge across the package**: this document ships `tools/provision.py` and `tools/verify_stream.py`; other documents name `eegtest`, `prov_eeg.py` and `tools/eegtest/ota.py` for the same two programs | A test step that names a binary that does not exist cannot be executed as written | programme | before the first production run |

| # | Item | Consequence | Owner | Gate |
|---|---|---|---|---|
| 13 | The **6.81 MB FAT partition holds about 71 s of audio** at 48 kHz 16-bit mono (calculated). Longer block audio must come from the card | F-11's pre-load model needs a decision | programme | Phase 2 |
| 14 | Secure-boot v2 bootloaders are larger than plain ones; the **partition table sits at `0x8000`**, leaving 32 kB | If the signed bootloader overflows, `CONFIG_PARTITION_TABLE_OFFSET` must move and every offset in section 9 changes. Phase 1 does not hit this, because Phase 1 builds no signed bootloader | firmware volunteer | first secure build, Phase 2 |
| 15 | **The E-29 acoustic clamp is not implemented.** `SET_HP_LEVEL` must clamp the codec volume register at the calibrated value; the codec driver is a stub | Calculated full-scale output is about 110 dB SPL against E-29's 100 dB SPL limit, so the requirement is **not met** until both the driver and the clamp exist | firmware volunteer | T17 and the E-29 type test |
| 16 | **No safety engineer has reviewed this design, and no hardware has been built.** *Corrected 2026-09-02: this row said "nothing in this package has been built". The firmware has been built and run under an emulator; no board has been fabricated and no unit exists* | Blocks use on a person. Does not block a build, a quote or firmware bring-up | programme | RISK-EEG-011 |

| # | Item | Consequence | Owner | Gate |
|---|---|---|---|---|
| 17 | **Static IRAM is 16,383 of 16,384 bytes used – one byte free** (section 9). Turning off the unused SPI-slave and gptimer ISRs did not move the figure by a single byte, so it is not those | The next function marked `IRAM_ATTR` fails the link with an error naming a section and not a cause, and E-13's tone scheduler and E-12's onset detector are both still to be written. Either the pool is genuinely full and someone must choose what leaves IRAM, or `esp_idf_size` is reporting against a fixed 16 kB window that is not the real limit on an ESP32-S3 with octal SPIRAM and XIP. Deciding which needs the linker map read against hardware; it is not guessed at from here | firmware volunteer | the linker map read on a unit, before the next IRAM function is added. Also SIM-EEG-018 open item 1 |
| 18 | **`CMD_ENTER_PROV` answers `0x02` for "already provisioned"**, where section 6.2 assigns `0x02` to bad length and `0x09` to locked – and `0x4B`, in the same handler, answers `0x09` for the same condition (section 6.2) | An operator reading the status table alone hunts a framing fault that is not there. `provision.py` hints both readings at step 1, which is a workaround, not a fix. `handle_provision()` also returns `0x10`-`0x18` and `0x20` codes that section 6.2 does not define at all | firmware volunteer (`main.c` is not this document's to edit) | one line in `main.c`, before T6 is first executed |

| # | Item | Consequence | Owner | Gate |
|---|---|---|---|---|
| 19 | **The abort-on-no-PSRAM is right for a bench and wrong for a field unit** (section 8.2). A unit that aborts in a participant's home cannot be told apart from a dead battery or a bad cable, because it never enumerates and the browser tool cannot ask it what is wrong | The FW-D13 guard is a real improvement over asserting inside `xRingbufferSend`, and it is still the wrong failure mode in the field. What a field unit should do – enumerate, report the fault in STATUS, refuse to record – is not written | firmware volunteer | before the first unit leaves the programme |
| 20 | **`provision.py` and `README_provisioning.md` still describe the old NVS layout** – namespace `"tiov"` in the default `nvs` partition, with an unconditional erase – which `drivers.c` no longer does (sections 2.5 and 7.3). Their `0x4A`/`0x4B` skip hints are also transposed: the config-write step reports a missing `0x4A` and the calibration-read step a missing `0x4B` | An operator following the script's own header would look for the constants in the wrong partition, and a skipped step would name the wrong opcode | firmware volunteer (those files are not this document's to edit) | before the first production run |

### Gap closure

This document closes the following entries from the v1 audit: `fw-project-skeleton-missing` (section 2), `fw-source-defect-register` (section 1.3), `fw-sdkconfig-defaults` (2.4 and 5.8), `fw-partition-table` (2.5), `fw-pinmap-gpio35-37-conflict` (section 3), `fw-driver-stubs` (1.2, with each stub bound to a test step), `icd-command-and-frame-payloads` (sections 5 and 6), `usb-identity-and-vid-pid` (4.3), `provisioning-script-f18` (section 7), `secure-boot-key-custody` (7.5), `firmware-release-package` (section 9), `eol-flashing-route` (2.3, 7.1 and 9), `host-production-test-tool` (section 8), and the microSD file layout, which section 5.9 now defines for the whole package. It leaves open, and says so above: `calibration-record-schema` (CAL-EEG-012 owns the schema; section 7.4 fixes only the identity block), the host tool naming, and the block-audio storage decision.